

SANTIAGO ENRIQUE VILLABONA APONTE



**TRABAJO DE GRADO PRESENTADO PARA OPTAR AL
TÍTULO DE**

INGENIERO DE SISTEMAS E INFORMÁTICA

**DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN
INTEGRADA DE INVENTARIO, VENTAS Y CLIENTES EN PEQUEÑOS
NEGOCIOS**

**ESCUELA DE INGENIERÍAS
INGENIERÍA DE SISTEMAS E INFORMÁTICA**

BUCARAMANGA

2026

DESARROLLO DE UNA APLICACIÓN WEB PARA LA GESTIÓN INTEGRADA DE INVENTARIO, VENTAS Y CLIENTES EN PEQUEÑOS NEGOCIOS

SANTIAGO ENRIQUE VILLABONA APONTE

Trabajo de grado presentado para optar al título de
INGENIERO DE SISTEMAS E INFORMÁTICA

Director:

ING. DANITH PATRICIA SOLORZANO ESCOBAR

UNIVERSIDAD PONTIFICIA BOLIVARIANA
ESCUELA DE INGENIERÍAS
INGENIERÍA DE SISTEMAS E INFORMÁTICA
BUCARAMANGA

2026

DEDICATORIA

Texto de la dedicatoria

Nombre del autor

AGRADECIMIENTOS

Texto de los agradecimientos

CONTENIDO

I. INTRODUCCIÓN	9
II. DELIMITACIÓN DEL PROBLEMA	10
A. Situación problema	10
III. ANTECEDENTES	12
IV. JUSTIFICACIÓN	14
A. Beneficios operativos para el negocio.....	14
B. Valor académico	15
C. Impacto social	15
D. Viabilidad técnica	16
V. OBJETIVOS	17
A. Objetivo general	17
B. Objetivos específicos	17
VI. MARCO TEÓRICO	18
A. Marco Conceptual.....	18
1) Arquitectura de Software	18
2) API REST (Representational State Transfer)	19
3) Gestión de Inventarios.....	19
4) Punto de Venta (POS - Point of Sale)	20
5) Base de Datos Relacional	20
B. Marco Tecnológico.....	21
1) Tecnologías Principales.....	21
2) Herramientas de Apoyo y Bibliotecas Complementarias	22
3) Justificación de la Selección Tecnológica.....	23

VII. DISEÑO DEL SISTEMA	25
A. Diagramas de Casos de Uso	25
B. Diagramas de Componentes	33
C. Diagramas de Secuencia.....	39
D. Resumen del Diseño.....	45
VIII. METODOLOGÍA	46
A. Enfoque Metodológico.....	46
B. Fases del Proyecto.....	46
IX. PRESUPUESTO	50
A. Notas sobre el Presupuesto	50
X. RESULTADOS	52
A. Requerimientos del Sistema	52
B. Requerimientos Funcionales.....	52
C. Requerimientos No Funcionales.....	77
D. Ejemplo de una lista	87
E. Ejemplo de una figura	87
XI. CONCLUSIONES.....	88
BIBLIOGRAFÍA	90

LISTA DE TABLAS

TABLA I	Cronograma de Ejecución del Proyecto	49
TABLA II	Distribución Semanal de Actividades	49
TABLA III	Presupuesto Estimado del Proyecto	50
TABLA IV	RF01: Autenticación y control de sesión	53
TABLA V	RF02: Gestión del catálogo de productos	54
TABLA VI	RF03: Registro de movimientos de inventario	55
TABLA VII	RF04: Alertas de stock bajo y umbral mínimo	56
TABLA VIII	RF05: Procesamiento de venta en tiempo real	57
TABLA IX	RF06: Gestión de métodos de pago y cálculo de cambio.....	58
TABLA X	RF07: Actualización automática de stock por venta	59
TABLA XI	RF08: Emisión de comprobantes de venta	60
TABLA XII	RF09: Gestión de anulaciones y devoluciones	61
TABLA XIII	RF10: Gestión del directorio de clientes.....	62
TABLA XIV	RF11: Consulta de historial de compras por cliente	63
TABLA XV	RF12: Dashboard de indicadores y reporte histórico filtrable.....	64
TABLA XVI	RF13: Gestión de categorías de productos	65
TABLA XVII	RF14: Gestión de imágenes de productos.....	66
TABLA XVIII	RF15: Búsqueda y filtrado avanzado de productos	67
TABLA XIX	RF16: Aplicación de descuentos en ventas	68
TABLA XX	RF17: Gestión del carrito de compras temporal	69
TABLA XXI	RF18: Consulta de ventas por rango de fechas	70
TABLA XXII	RF19: Registro de múltiples contactos por cliente.....	71
TABLA XXIII	RF20: Segmentación de clientes por criterios.....	72
TABLA XXIV	RF21: Reporte de productos más vendidos.....	73
TABLA XXV	RF22: Análisis de rentabilidad por producto.....	74

TABLA XXVI	RF23: Reporte de ventas por método de pago	75
TABLA XXVII	RF24: Reporte de movimientos de inventario	76
TABLA XXVIII	RNF01: Protección de credenciales y datos sensibles.....	77
TABLA XXIX	RNF02: Control de acceso y gestión de sesiones de usuario	78
TABLA XXX	RNF03: Rendimiento en operaciones críticas de negocio	79
TABLA XXXI	RNF04: Interfaz intuitiva y de fácil aprendizaje	80
TABLA XXXII	RNF05: Adaptabilidad a diferentes dispositivos y tamaños de pantalla	81
TABLA XXXIII	RNF06: Escalabilidad de datos y modularidad arquitectónica	82
TABLA XXXIV	RNF07: Integridad transaccional y consistencia de la información	83
TABLA XXXV	RNF08: Disponibilidad del servicio y recuperación ante fallos	84
TABLA XXXVI	RNF09: Mantenibilidad del código mediante modularidad, pruebas y documentación.....	85
TABLA XXXVII	RNF10: Auditoría y registro de eventos críticos.....	86

GLOSARIO

SIGLA	SIGNIFICADO
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
ERP	Enterprise Resource Planning
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
IP	Internet Protocol
IVA	Impuesto al Valor Agregado
JWT	JSON Web Token
MVP	Minimum Viable Product
ORM	Object-Relational Mapping
POS	Point of Sale
REST	Representational State Transfer
SKU	Stock Keeping Unit
SQL	Structured Query Language
SSG	Static Site Generation
SSR	Server-Side Rendering
UI	User Interface
UX	User Experience

I. INTRODUCCIÓN

Este proyecto de grado propone desarrollar una aplicación web que permita a pequeños negocios colombianos gestionar su inventario, ventas, clientes y reportes desde un solo lugar. La idea nace de una realidad que se observa a diario en tiendas, papelerías y negocios de barrio: muchos propietarios llevan sus cuentas en cuadernos, hojas de Excel o simplemente de memoria, y cuando necesitan saber cuánto vendieron en el mes o qué producto se está agotando, no tienen cómo saberlo con certeza.

Existen soluciones comerciales como Odoo, Alegra o Siigo, pero sus costos de suscripción, su complejidad de uso o su orientación a empresas más grandes las hacen poco viables para un negocio que apenas empieza o que opera con márgenes ajustados. Este proyecto busca ofrecer una alternativa accesible, construida con tecnologías web actuales y de código abierto.

El sistema se diseña con una arquitectura por capas: un frontend en Next.js para la interfaz de usuario, un backend en NestJS que contiene la lógica de negocio, y PostgreSQL como base de datos. El objetivo no es solo construir una herramienta funcional, sino hacerlo aplicando prácticas de ingeniería de software que demuestren lo aprendido durante la carrera.

El documento está organizado de la siguiente manera. Primero se describe el problema que motiva el proyecto y se plantea la pregunta de investigación. Luego se definen los objetivos, se revisan soluciones existentes en la sección de antecedentes, y se justifica la pertinencia del proyecto. El marco teórico establece los conceptos y tecnologías necesarios para entender la propuesta. La metodología explica cómo se llevará a cabo el desarrollo en 12 semanas, y la sección de diseño del sistema presenta la arquitectura y los diagramas que guiarán la implementación.

Con este trabajo se busca demostrar competencias concretas en análisis de requisitos, diseño de software, implementación web y validación con usuarios, aplicando de forma integrada los conocimientos adquiridos en la formación como Ingeniero de Sistemas.

II. DELIMITACIÓN DEL PROBLEMA

A. *Situación problema*

En Colombia, según cifras del DANE, las micro y pequeñas empresas representan más del 90 % del tejido empresarial. Sin embargo, una proporción considerable de estos negocios gestiona sus operaciones diarias con herramientas rudimentarias: cuadernos donde se anotan las ventas del día, hojas de cálculo que no se actualizan con regularidad, o simplemente la memoria del propietario. Esta forma de operar, aunque comprensible dada la falta de alternativas accesibles, genera problemas concretos que se acumulan con el tiempo.

El primero y más visible es el descontrol del inventario. Cuando un tendero no sabe con exactitud qué productos tiene, cuántos quedan ni cuánto valen en conjunto, termina comprando mercancía que ya tiene o dejando de pedir la que sus clientes buscan. Ambos escenarios le cuestan dinero: el exceso de stock inmoviliza capital, y la falta de producto significa ventas perdidas.

El segundo problema es la ausencia de registro confiable de ventas. Si las transacciones se anotan en un cuaderno o no se anotan, resulta muy difícil saber cuánto se vendió en la semana, cuál fue el producto más popular o si un cliente tiene pagos pendientes. Sin estos datos, las decisiones del negocio se toman por intuición, no por información.

El tercer problema tiene que ver con los clientes. Los negocios pequeños conocen a sus clientes frecuentes de cara, pero no llevan un registro organizado de quiénes son, qué compran ni con qué frecuencia vuelven. Esto impide hacer seguimiento, ofrecer descuentos a clientes fieles o detectar cambios en los hábitos de compra.

Por último, la generación de reportes es prácticamente inexistente. Para saber cómo le fue al negocio en el mes, el propietario tendría que sentarse a sumar cifras de distintas fuentes, un proceso que puede tomar horas y que rara vez se hace. En la práctica, muchos propietarios no conocen con precisión sus márgenes de ganancia ni sus tendencias de venta.

Frente a esta situación, el mercado ofrece soluciones como Odoo, Alegra o Siigo. Son plataformas completas, pero presentan limitaciones para el público al que apunta este proyecto. Sus planes de pago representan un gasto mensual que muchos microempresarios no pueden asumir, especialmente cuando el negocio está empezando. Además, varias de estas herramientas están pensadas para empresas con personal administrativo o conocimientos contables, lo que genera una curva de aprendizaje que desalienta su uso. Las opciones gratuitas, por su parte, suelen venir en

inglés, con procesos diseñados para otros mercados y con funcionalidades limitadas que obligan a migrar a planes pagos una vez que el negocio crece.

Ante este escenario, surge la oportunidad de construir una aplicación web de código abierto, en español, pensada desde el inicio para las necesidades y limitaciones de un pequeño negocio colombiano. Una herramienta que integre inventario, ventas, clientes y reportes en un solo sistema accesible desde cualquier navegador. Con base en esta problemática, se plantea la siguiente pregunta de investigación:

¿Cómo desarrollar una aplicación web que permita a pequeños negocios gestionar de manera integrada sus procesos de inventario, ventas, clientes y reportes, mediante una arquitectura modular por capas que garantice organización, mantenibilidad y escalabilidad?

III. ANTECEDENTES

Los pequeños negocios que quieren organizar sus inventarios y ventas con tecnología se encuentran con un mercado donde las opciones disponibles no siempre se ajustan a su realidad. En esta sección se revisan tres plataformas comerciales reconocidas en el contexto colombiano y latinoamericano, y se mencionan trabajos académicos previos que han abordado problemas similares.

Odoo Inventory. Odoo es una plataforma de origen belga que ofrece módulos integrados para distintas áreas del negocio: inventario, ventas, compras, contabilidad, entre otros. Su módulo de inventario permite gestionar productos, rastrear movimientos de stock, configurar alertas de inventario bajo y generar órdenes de reabastecimiento de forma automática. Según su sitio oficial, Odoo cuenta con más de 12 millones de usuarios en el mundo [1]. La plataforma ofrece una versión Community de código abierto con funcionalidades básicas y una versión Enterprise de pago. El problema para un negocio pequeño colombiano es que la versión gratuita tiene limitaciones importantes (por ejemplo, no incluye módulo de punto de venta completo), y la versión de pago parte desde aproximadamente 24 USD por usuario al mes. Para una tienda de barrio con ingresos variables, ese costo mensual recurrente no es trivial.

Alegra. Esta plataforma colombiana se especializa en facturación electrónica y gestión contable [2]. Integra módulos de inventario, facturación conforme a los requisitos de la DIAN, y control de ventas. Su principal ventaja es que fue diseñada para el mercado colombiano: incluye cálculo de IVA según la normativa local y formatos de facturación electrónica homologados. Sin embargo, su plan gratuito permite un máximo de 15 facturas por mes y restringe el acceso a reportes y funcionalidades de inventario avanzadas. Los planes de pago arrancan en \$49.900 COP mensuales (aproximadamente 12 USD), lo que puede ser un gasto considerable para un negocio que vende montos pequeños. Además, Alegra está pensada para usuarios con al menos nociones básicas de contabilidad, lo que no siempre es el caso de los propietarios de micronegocios.

Siigo. Es otro sistema colombiano enfocado en contabilidad y facturación electrónica [3]. Ofrece módulos de inventario y ventas que se sincronizan con su núcleo contable, y genera reportes financieros que cumplen con requisitos tributarios. Su fortaleza está en la profundidad contable, pero eso mismo es su limitación para el público objetivo de este proyecto: Siigo asume que el usuario entiende conceptos como cuentas contables, centros de costo y conciliaciones bancarias. Un tendero que solo necesita saber cuántos productos le quedan y cuánto vendió hoy se enfrenta

a una interfaz que le pide información que no maneja. Al igual que las anteriores, Siigo opera con suscripciones mensuales que deben evaluarse frente a los ingresos del negocio.

Trabajos académicos previos. En universidades colombianas y latinoamericanas se han desarrollado proyectos de grado que abordan esta misma problemática. Algunos se han enfocado en sectores específicos sistemas de inventario para farmacias, ferreterías o restaurantes y otros han explorado arquitecturas de software para aplicaciones de gestión empresarial. La revisión de estos trabajos permitió identificar lecciones útiles: la importancia de acotar el alcance a funcionalidades esenciales para evitar proyectos que no se completan, la ventaja de usar frameworks con buena documentación para reducir tiempos de desarrollo, y la necesidad de validar el sistema con usuarios reales antes de considerar que cumple su objetivo.

En resumen, las soluciones comerciales existentes son herramientas maduras y completas, pero no están diseñadas para el microempresario colombiano que necesita algo simple, en español y sin costo mensual. Los trabajos académicos previos confirman que es un problema válido y aportan experiencias sobre qué funciona y qué no al construir este tipo de sistemas. El presente proyecto se diferencia al combinar un alcance acotado (inventario, ventas, clientes, reportes), tecnologías web actuales de código abierto y un enfoque de desarrollo orientado a demostrar competencias de ingeniería de software.

IV. JUSTIFICACIÓN

Este proyecto se justifica desde tres perspectivas: los beneficios operativos para el negocio, el valor académico para la formación del estudiante y la viabilidad técnica de la propuesta.

A. *Beneficios operativos para el negocio*

El problema central que se busca resolver es la dispersión de información. Un propietario que anota ventas en un cuaderno, lleva el inventario en una hoja de Excel y guarda los datos de sus clientes en el teléfono tiene su información repartida en tres lugares que no se comunican entre sí. Si quiere saber cuánto vendió esta semana o cuántos productos le quedan, tiene que cruzar datos manualmente. Eso toma tiempo y es propenso a errores.

La aplicación propuesta centraliza toda esa información en un solo sistema. Los beneficios concretos son:

- **Inventario en tiempo real.** Cada vez que se registra una venta, el stock se actualiza automáticamente. El propietario puede consultar en cualquier momento qué productos tiene, cuáles se están agotando y cuál es el valor total de su mercancía.
- **Registro de ventas con trazabilidad.** Cada transacción queda guardada con su fecha, productos vendidos, montos y método de pago. Esto permite saber cuáles son los productos más vendidos, en qué días se vende más y si hay pagos pendientes.
- **Gestión de clientes.** Tener un registro organizado de clientes frecuentes y su historial de compras permite identificar patrones, hacer seguimiento y tomar decisiones sobre qué productos ofrecer.
- **Reportes automáticos.** Lo que antes tomaba horas de cálculos manuales ventas del mes, productos más vendidos, ganancias por período se genera con un clic y se presenta en gráficas claras.
- **Reducción de errores.** El cálculo automático de totales, IVA y niveles de stock elimina los errores que ocurren cuando se hacen estas operaciones a mano, especialmente en días de mucho movimiento.
- **Acceso desde cualquier dispositivo.** Al ser una aplicación web, funciona desde cualquier computador, tablet o celular con navegador. No requiere instalar software.

B. Valor académico

Más allá de la herramienta en sí, este proyecto es una oportunidad para aplicar de forma integrada lo aprendido durante la carrera de Ingeniería de Sistemas. No se trata solo de escribir código, sino de pasar por todas las etapas de un proyecto de software real:

- **Arquitectura de software.** Diseñar un sistema con separación de capas (presentación, lógica de negocio, datos) y aplicar patrones como Repository, Inyección de Dependencias y DTO.
- **Desarrollo web completo.** Construir tanto el frontend (Next.js) como el backend (NestJS) e integrarlos mediante una API REST. El alcance se limita a una aplicación web accesible desde navegador; no incluye desarrollo de aplicaciones móviles nativas.
- **Bases de datos.** Diseñar un modelo relacional normalizado, escribir consultas eficientes y gestionar la persistencia con Prisma ORM y PostgreSQL.
- **Ingeniería de requisitos.** Identificar las necesidades del usuario, definir requisitos funcionales y no funcionales, y documentar casos de uso.
- **Gestión de proyecto.** Organizar el trabajo en sprints semanales, usar un tablero Kanban para el seguimiento de tareas y mantener control de versiones con Git.
- **Pruebas de software.** Implementar pruebas unitarias, pruebas de integración y validación con usuarios reales.

C. Impacto social

Si bien este es un proyecto académico, su diseño apunta a un problema real. Una herramienta gratuita y en español que permita a un tendero de barrio organizar su negocio tiene un impacto potencial concreto: le da acceso a capacidades que hoy solo están disponibles en plataformas de pago. Un negocio que lleva registros ordenados puede demostrar ingresos ante una entidad financiera, tomar mejores decisiones de compra y detectar a tiempo cuándo algo no está funcionando. Esto contribuye tanto a la formalización de la economía como a la competitividad de los pequeños negocios.

D. Viabilidad técnica

El proyecto es técnicamente viable por varias razones. Todas las tecnologías seleccionadas son de código abierto, lo que elimina costos de licenciamiento. Cuentan con documentación amplia y comunidades activas donde la mayoría de problemas técnicos ya han sido resueltos y documentados. El despliegue puede hacerse de forma gratuita en plataformas como Vercel (frontend) y Railway (backend), lo que permite tener el sistema funcionando en línea sin costo de infraestructura. El alcance se ha delimitado a cuatro módulos esenciales (inventario, ventas, clientes, reportes), lo que mantiene el proyecto manejable dentro de las 12 semanas disponibles.

V. OBJETIVOS

A. *Objetivo general*

Desarrollar una aplicación web para la gestión integrada de inventario, ventas, clientes y reportes, mediante una arquitectura modular por capas (presentación, lógica de negocio y acceso a datos) implementada con tecnologías de desarrollo web actuales como Next.js, NestJS, PostgreSQL y TypeScript, con el fin de permitir a pequeños negocios colombianos centralizar y controlar sus procesos operativos y administrativos de forma eficiente y accesible.

B. *Objetivos específicos*

- Diseñar la estructura del sistema mediante capas de presentación, lógica de negocio y acceso a datos, estableciendo los módulos funcionales (inventario, ventas, clientes, reportes), sus responsabilidades y mecanismos de comunicación, garantizando separación de responsabilidades y escalabilidad.
- Desarrollar la capa de servidor mediante una arquitectura RESTful (Representational State Transfer) que implemente la lógica de negocio de los módulos de inventario, ventas, clientes y reportes, garantizando la persistencia de datos, validación de reglas de negocio y exposición de servicios mediante endpoints documentados.
- Implementar la capa de presentación mediante componentes reutilizables que permitan la interacción del usuario con los módulos del sistema, implementando formularios, tablas, filtros y visualizaciones de datos, priorizando la usabilidad y accesibilidad desde diferentes dispositivos.
- Verificar el correcto funcionamiento de los módulos implementados mediante pruebas de integración, pruebas de casos de uso completos y evaluación con usuarios potenciales, documentando hallazgos y realizando ajustes necesarios para garantizar la calidad del sistema.

VI. MARCO TEÓRICO

Esta sección presenta los conceptos teóricos y las tecnologías que sustentan el proyecto. El marco conceptual cubre los fundamentos de arquitectura de software, APIs, gestión de inventarios y bases de datos. El marco tecnológico describe las herramientas concretas elegidas para la implementación y justifica su selección.

A. *Marco Conceptual*

1) *Arquitectura de Software:* La arquitectura de software define cómo se organiza un sistema a alto nivel: qué partes lo componen, cómo se relacionan entre sí y bajo qué reglas se comunican. Tener una arquitectura clara desde el inicio del proyecto permite que el código sea más fácil de entender, de modificar y de mantener a medida que crece.

La arquitectura por capas es uno de los patrones más utilizados en la industria. Consiste en dividir el sistema en niveles horizontales, donde cada nivel tiene una responsabilidad concreta y solo se comunica con los niveles inmediatamente adyacentes. Las ventajas principales de este enfoque son: cada capa se ocupa de un problema específico sin mezclarse con los demás (separación de responsabilidades); un cambio en una capa no debería afectar al resto si se respeta la interfaz de comunicación (bajo acoplamiento); y es posible reemplazar una capa completa por ejemplo, cambiar de base de datos sin reescribir las demás.

Es importante distinguir entre la arquitectura general del sistema y los patrones de diseño específicos que se aplican dentro de ella. La arquitectura por capas no es equivalente al patrón MVC (Modelo-Vista-Controlador), aunque comparten el principio de separación de responsabilidades. MVC organiza el código en tres roles fijos (modelo, vista y controlador) dentro de una misma aplicación, mientras que la arquitectura por capas divide el sistema en niveles independientes que pueden desplegarse y evolucionar por separado. En el contexto de este proyecto, la arquitectura por capas resulta más adecuada que MVC porque refleja la separación física entre el frontend (Next.js) y el backend (NestJS), cada uno ejecutándose como una aplicación independiente que se comunica mediante API REST.

En el contexto específico del presente proyecto, se implementa una arquitectura de tres capas claramente diferenciadas. La capa de presentación, implementada con Next.js, se encarga de la interfaz de usuario: muestra información, captura entradas del usuario y presenta los datos de forma comprensible. La capa de lógica de negocio, implementada con NestJS, contiene el procesamiento

de datos y las reglas que definen cómo opera el sistema; orquesta operaciones, valida datos y coordina la interacción entre módulos. La capa de acceso a datos gestiona la persistencia y recuperación de información hacia y desde PostgreSQL, abstrayendo los detalles de almacenamiento del resto del sistema.

Dentro de esta arquitectura se aplican patrones de diseño específicos que resuelven problemas concretos de implementación. El patrón Repository se utiliza a través de Prisma ORM para encapsular la lógica de acceso a datos y desacoplarla de la lógica de negocio. La Inyección de Dependencias, que NestJS implementa de forma nativa mediante su contenedor IoC, permite que los módulos reciban sus dependencias sin crearlas directamente, lo que facilita las pruebas y el mantenimiento. El patrón DTO (Data Transfer Object) se emplea para validar y tipar los datos que viajan entre capas, asegurando que cada capa reciba solo la información que necesita en el formato esperado. Finalmente, el patrón Module organiza el backend en módulos cohesivos (inventario, ventas, clientes, reportes), cada uno agrupando sus controladores, servicios y repositorios relacionados.

2) API REST (*Representational State Transfer*): REST es un estilo de arquitectura para comunicar aplicaciones a través de la web. En la práctica, una API REST usa los métodos HTTP que ya existen GET para consultar, POST para crear, PUT para actualizar y DELETE para eliminar sobre URLs que identifican recursos concretos (por ejemplo, `/api/productos/123`).

Los principios que definen REST son pocos pero importantes. Cada petición debe ser independiente: el servidor no guarda estado entre peticiones (stateless), lo que simplifica la escalabilidad. Cliente y servidor están separados y pueden evolucionar sin afectarse mutuamente. Las respuestas pueden cachearse para mejorar el rendimiento. Y la interfaz es uniforme: los mismos métodos HTTP y convenciones de URL se aplican a todos los recursos.

En este proyecto, la API REST es el canal de comunicación entre el frontend (Next.js) y el backend (NestJS). Cada operación del sistema consultar productos, registrar una venta, buscar un cliente se realiza mediante una petición HTTP a un endpoint documentado. La documentación se genera de forma automática con Swagger, lo que permite probar los endpoints desde el navegador sin herramientas externas.

3) Gestión de Inventarios: La gestión de inventarios consiste en llevar un control organizado de los productos que tiene un negocio: qué hay en stock, cuánto entra, cuánto sale y cuánto vale. En un sistema de información, esto se traduce en cuatro funciones concretas. El control de

stock registra cada entrada (compras, ajustes) y cada salida (ventas, mermas) para mantener actualizada la cantidad disponible de cada producto. La valorización calcula el valor total del inventario considerando precios de costo y de venta, lo cual es útil para saber cuánto dinero tiene el negocio invertido en mercancía. La trazabilidad permite consultar el historial de movimientos de un producto: cuándo entró, cuándo se vendió, si hubo ajustes. Las alertas avisan cuando un producto baja de un umbral mínimo definido por el usuario, para que el propietario pueda reponerlo antes de quedarse sin existencias.

Para un negocio pequeño, tener este control significa evitar dos problemas frecuentes: quedarse sin un producto que los clientes buscan (venta perdida) o acumular mercancía que no rota (capital inmovilizado). Con datos reales de movimiento, el propietario puede decidir qué comprar, en qué cantidad y con qué frecuencia, en lugar de hacerlo por intuición.

4) Punto de Venta (POS - Point of Sale): Un sistema de punto de venta (POS) es la pantalla desde la que se registra cada venta. En su forma más básica, permite buscar productos, agregarlos a un carrito, calcular el total con impuestos, registrar el método de pago y generar un comprobante. Es el punto donde convergen el inventario (porque cada venta descuenta stock) y los clientes (porque se puede asociar la compra a un cliente registrado).

En un negocio pequeño, el POS tiene que ser rápido el cliente no puede esperar mientras el sistema carga e intuitivo, de modo que cualquier persona pueda operar la caja después de una explicación breve. La reducción automática de stock al completar la venta es fundamental, porque si el inventario no se actualiza en tiempo real, los datos pierden confiabilidad.

5) Base de Datos Relacional: Una base de datos relacional organiza la información en tablas que se conectan entre sí mediante claves primarias y foráneas. Se consulta y manipula con SQL (Structured Query Language), un lenguaje estándar que permite desde consultas simples hasta operaciones complejas con joins, agregaciones y filtros.

Para este proyecto, el modelo relacional es adecuado por varias razones. La integridad referencial impide, por ejemplo, que exista una venta asociada a un cliente que ya fue eliminado. Las transacciones ACID garantizan que una operación compuesta como registrar una venta y descontar stock simultáneamente se ejecute completa o no se ejecute en absoluto, evitando estados inconsistentes. Las consultas SQL permiten generar reportes que crucen datos de ventas, productos y clientes sin necesidad de programación adicional. La normalización del modelo reduce la duplica-

ción de datos: la información de un producto se guarda una sola vez y se referencia desde las tablas de ventas e inventario.

B. Marco Tecnológico

A continuación se describen las tecnologías seleccionadas para implementar el sistema. Todas son de código abierto, cuentan con documentación amplia y fueron elegidas considerando su madurez, la actividad de su comunidad y su compatibilidad entre sí.

1) Tecnologías Principales: **Next.js** es el framework usado para construir el frontend. Está basado en React, pero agrega funcionalidades que React solo no tiene: renderizado del lado del servidor (SSR), que permite generar las páginas en el servidor antes de enviarlas al navegador; un sistema de rutas basado en archivos, donde la estructura de carpetas define automáticamente las URLs de la aplicación; optimización automática de imágenes y código; y soporte nativo para TypeScript. Su App Router y los Server Components (introducidos en la versión 13 y mejorados en la 14) permiten ejecutar parte del código React directamente en el servidor, reduciendo el JavaScript que el navegador debe descargar.

NestJS es el framework para el backend. Está construido sobre Node.js y usa TypeScript de forma nativa. Su estructura se basa en módulos: cada módulo agrupa un controlador (que recibe las peticiones HTTP), un servicio (que contiene la lógica de negocio) y las entidades o repositorios necesarios. NestJS incluye inyección de dependencias de forma integrada, lo que permite que los componentes reciban sus dependencias sin crearlas directamente esto facilita las pruebas porque se pueden sustituir dependencias reales por mocks . También genera documentación Swagger de la API de forma automática a partir de decoradores en el código.

TypeScript es el lenguaje que se usa tanto en el frontend como en el backend. Extiende JavaScript con un sistema de tipos estático: cada variable, función y objeto tiene un tipo definido que el compilador verifica antes de ejecutar el código. En la práctica, esto significa que muchos errores se detectan mientras se escribe el código (en el editor) en lugar de descubrirlos al ejecutar la aplicación. El autocompletado que ofrecen los editores con TypeScript también acelera el desarrollo, porque sugiere métodos y propiedades disponibles según el tipo de cada objeto.

PostgreSQL es la base de datos relacional del proyecto. Es de código abierto, cumple con el estándar SQL, y soporta transacciones ACID, lo que garantiza que operaciones compuestas (como registrar una venta y descontar stock) se ejecuten de forma atómica. Soporta tipos de datos

avanzados como JSON, arrays y tipos personalizados, lo que permite modelar datos complejos sin sacrificar el modelo relacional. Sus índices y optimizador de consultas hacen que las operaciones de lectura y escritura sean eficientes incluso con volúmenes grandes de datos.

Prisma ORM es la capa que conecta el código TypeScript con la base de datos. En lugar de escribir SQL directamente, se define un esquema declarativo (archivo `schema.prisma`) que describe las tablas y sus relaciones. A partir de ese esquema, Prisma genera un cliente TypeScript con tipado completo: cada consulta tiene autocompletado y verificación de tipos en tiempo de compilación. Las migraciones se generan automáticamente al modificar el esquema, y Prisma Studio ofrece una interfaz visual para explorar los datos durante el desarrollo.

2) Herramientas de Apoyo y Bibliotecas Complementarias: Además de las tecnologías principales, el proyecto usa las siguientes herramientas de apoyo:

Control de versiones y gestión de tareas. Git con GitHub para el código fuente: cada cambio queda registrado, se trabaja con branches para funcionalidades nuevas y se integran mediante pull requests. Trello se usa como tablero Kanban para organizar las tareas semanales en columnas (pendiente, en progreso, terminado).

Diseño y entorno. Figma para crear wireframes y prototipos de la interfaz antes de implementarla. Docker para levantar la base de datos PostgreSQL en un contenedor local, garantizando que el entorno de desarrollo sea el mismo sin importar la máquina.

Despliegue. Vercel para el frontend (integración continua desde GitHub, CDN global, plan gratuito). Railway para el backend y la base de datos en producción (plan gratuito con capacidad suficiente para el proyecto).

Librerías del frontend. shadcn/ui (componentes de interfaz basados en Tailwind CSS y Radix UI), TanStack Query (manejo de estado del servidor y caché de datos), React Hook Form (gestión de formularios con validación), Zod (validación de esquemas en TypeScript) y Recharts (gráficas para reportes y dashboard).

Librerías del backend. class-validator y class-transformer (validación y transformación de DTOs con decoradores), Swagger con OpenAPI (documentación interactiva de la API generada a partir del código).

Imágenes. Cloudinary como servicio en la nube para almacenar y optimizar las imágenes de productos. Su plan gratuito ofrece 25 GB mensuales, suficiente para el alcance del proyecto.

Testing. Jest como framework de pruebas unitarias para backend y frontend. Supertest para pruebas de integración de endpoints HTTP. Opcionalmente, Playwright para pruebas de flujo completo en el navegador.

3) *Justificación de la Selección Tecnológica:* La elección del stack tecnológico se fundamenta en criterios verificables de vigencia, adopción y soporte activo. A continuación se presenta evidencia concreta de por qué cada tecnología seleccionada puede considerarse actual y adecuada para el proyecto:

- **Next.js** (creado en 2016 por Vercel): su versión 14, lanzada en octubre de 2023, introdujo Server Actions y mejoras sustanciales de rendimiento. Según la encuesta State of JavaScript 2023, Next.js es el framework de React con mayor adopción en producción. El paquete `next` supera los 6 millones de descargas semanales en npm.
- **NestJS** (creado en 2017 por Kamil Myśliwiec): actualmente en su versión 10, cuenta con más de 70.000 estrellas en GitHub y más de 3 millones de descargas semanales en npm. Es el framework de Node.js para backend con mayor crecimiento en adopción empresarial según encuestas de la comunidad.
- **PostgreSQL** (primera versión estable en 1996, con desarrollo continuo): su versión 16, lanzada en septiembre de 2023, incorporó mejoras en paralelismo de consultas y rendimiento de JSON. Es la base de datos relacional de código abierto más utilizada según el ranking DB-Engines 2024.
- **TypeScript** (creado por Microsoft en 2012): su versión 5.x introdujo decoradores nativos y mejoras de rendimiento. Según la encuesta Stack Overflow Developer Survey 2024, TypeScript se ubica entre los cinco lenguajes más utilizados y apreciados por desarrolladores.
- **Prisma ORM** (primera versión estable en 2021): diseñado específicamente para TypeScript, ofrece un cliente de base de datos con tipado seguro, migraciones automáticas y generación de tipos a partir del esquema.

Para poner en perspectiva la vigencia de estas tecnologías, basta contrastarlas con alternativas que hoy se consideran obsoletas o en desuso: PHP 5 (sin soporte desde 2018), jQuery

como framework principal de interfaz (reemplazado por frameworks reactivos), o MySQL 5.0 (sin muchas de las funcionalidades que PostgreSQL 16 ofrece nativamente, como tipos JSON, CTEs recursivos o búsqueda de texto completo). Todas las tecnologías seleccionadas para este proyecto reciben actualizaciones frecuentes, cuentan con hojas de ruta públicas y tienen comunidades activas que resuelven problemas y publican recursos educativos de forma continua.

El tipado estático proporcionado por TypeScript en todas las capas del sistema reduce errores de programación, mejora la experiencia de desarrollo mediante autocompletado y verificación en tiempo real, y facilita el mantenimiento del código a largo plazo.

La arquitectura modular que NestJS habilita de forma nativa facilita la implementación de una arquitectura por capas clara y escalable, adecuada para demostrar conceptos arquitectónicos fundamentales en un contexto académico. La compatibilidad nativa entre Next.js y React, NestJS y TypeScript, y Prisma con PostgreSQL reduce la complejidad de integración entre componentes.

Las plataformas de despliegue seleccionadas (Vercel para frontend, Railway para backend) ofrecen planes gratuitos con capacidades suficientes para el alcance académico del proyecto, lo que permite demostrar la aplicación funcionando en producción sin costos de infraestructura.

VII. DISEÑO DEL SISTEMA

Esta sección presenta el diseño detallado del sistema mediante diagramas UML que describen tanto las funcionalidades disponibles para los usuarios como la arquitectura técnica interna y los flujos de interacción entre componentes.

A. *Diagramas de Casos de Uso*

Los diagramas de casos de uso describen las funcionalidades del sistema desde la perspectiva de los usuarios, identificando los actores que interactúan con el sistema y las acciones que pueden realizar. El sistema define tres roles principales: Administrador, Cajero y Usuario de Inventario.

El módulo de autenticación (Figura 1) permite a todos los usuarios del sistema iniciar y cerrar sesión, recuperación de contraseñas, y gestión de perfiles. Los administradores tienen privilegios adicionales para registrar nuevos usuarios.

El módulo de productos (Figura 2) permite la gestión completa del inventario. Los administradores y usuarios de inventario pueden crear, editar, eliminar y consultar productos, gestionar categorías, controlar niveles de stock, y recibir alertas de stock bajo.

El módulo de ventas (Figura 3) representa el punto de venta (POS) del sistema. Los cajeros pueden procesar ventas, pausar y reanudar transacciones, gestionar métodos de pago, y generar comprobantes. Los administradores pueden además cancelar ventas y acceder a reportes detallados.

El módulo de clientes (Figura 4) permite gestionar la información de los clientes del negocio. Todos los roles pueden consultar clientes, mientras que los administradores y cajeros pueden registrar nuevos clientes y actualizar su información. Los administradores también pueden eliminar clientes (soft delete).

El módulo de reportes (Figura 5) proporciona acceso a métricas y estadísticas del negocio. Los administradores y cajeros pueden acceder al dashboard con KPIs en tiempo real, reportes de ventas, productos más vendidos, y tendencias. Los administradores también pueden exportar reportes en diferentes formatos.

El módulo de configuración (Figura 6) permite a los administradores gestionar usuarios del sistema, asignar roles, configurar parámetros del negocio, y ajustar configuraciones generales.

El módulo de categorías (Figura 7) permite organizar los productos en categorías jerárquicas, facilitando la búsqueda y gestión del inventario. Los administradores y usuarios de inventario

Fig. 1 Diagrama de Casos de Uso - Módulo de Autenticación

Fig. 2 Diagrama de Casos de Uso - Módulo de Productos

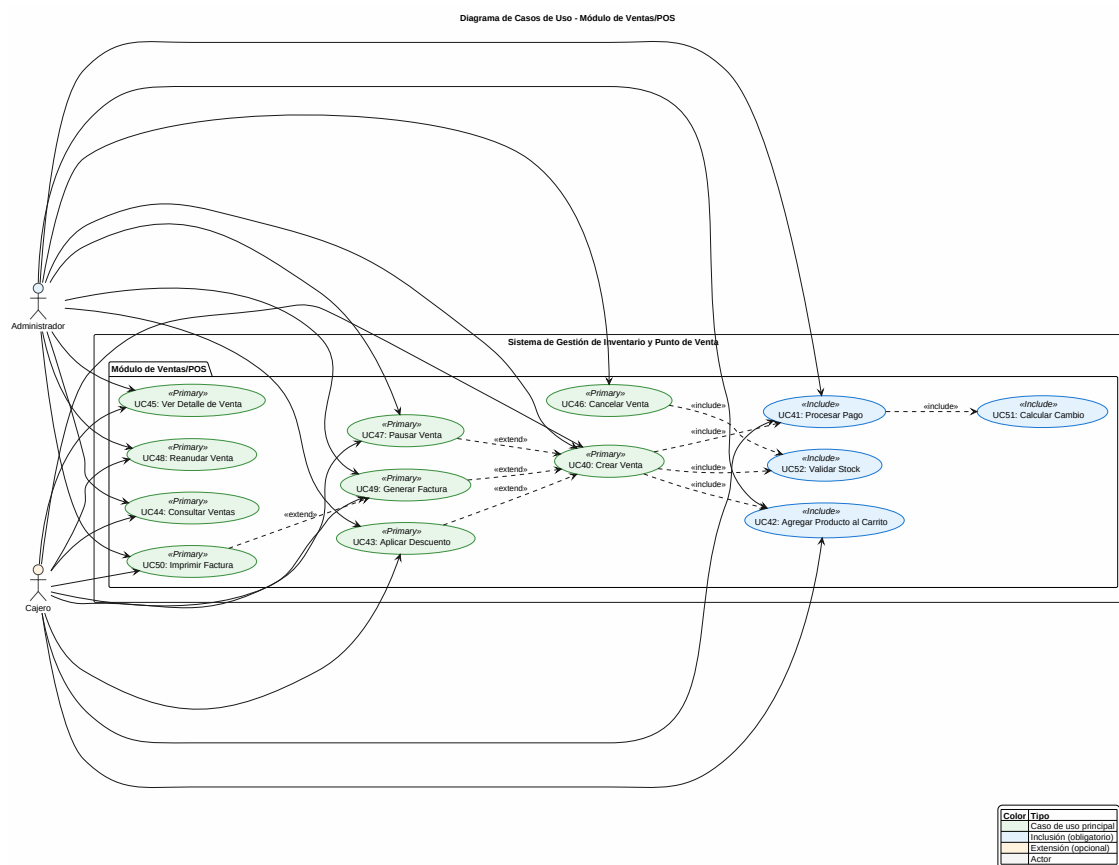


Fig. 3 Diagrama de Casos de Uso - Módulo de Ventas

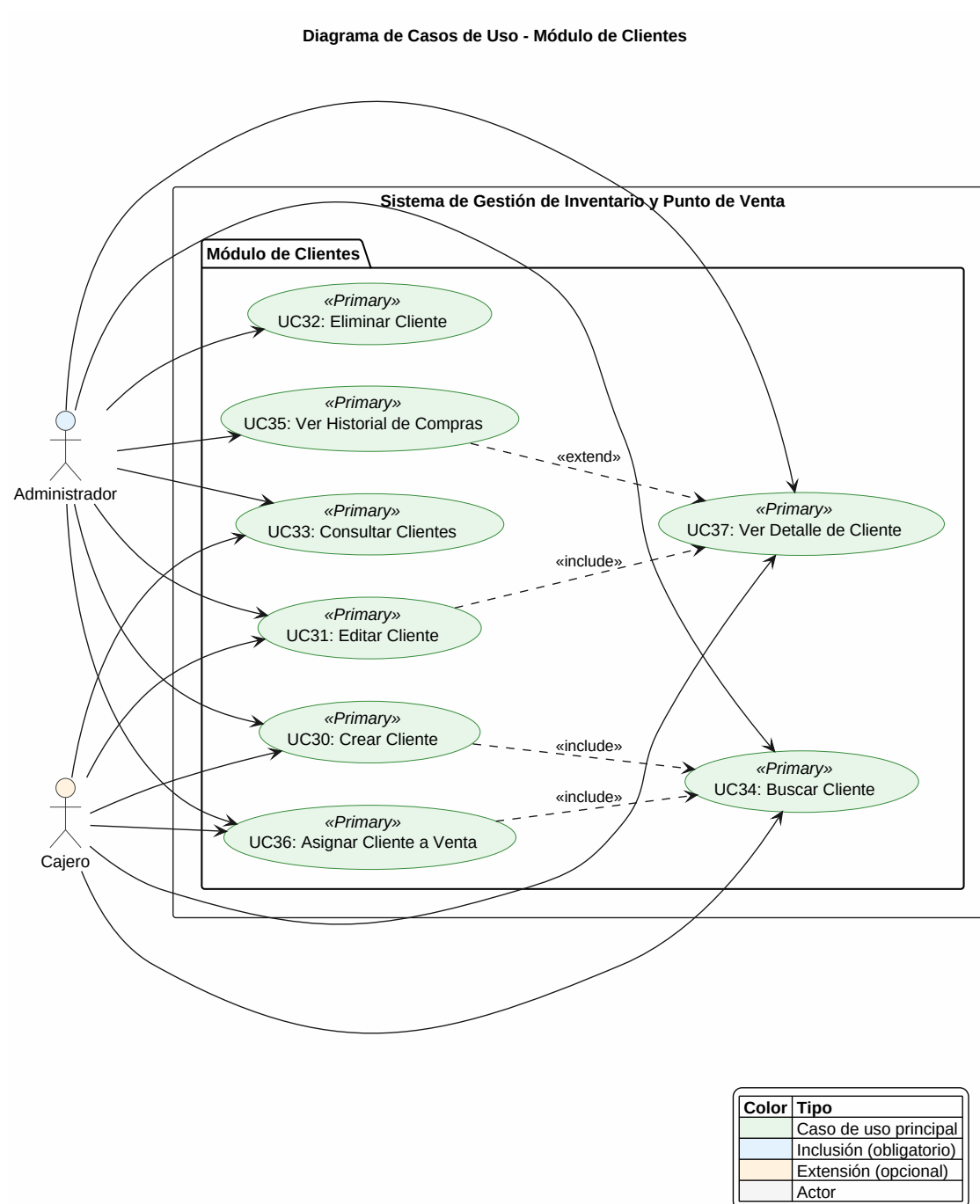


Fig. 4 Diagrama de Casos de Uso - Módulo de Clientes

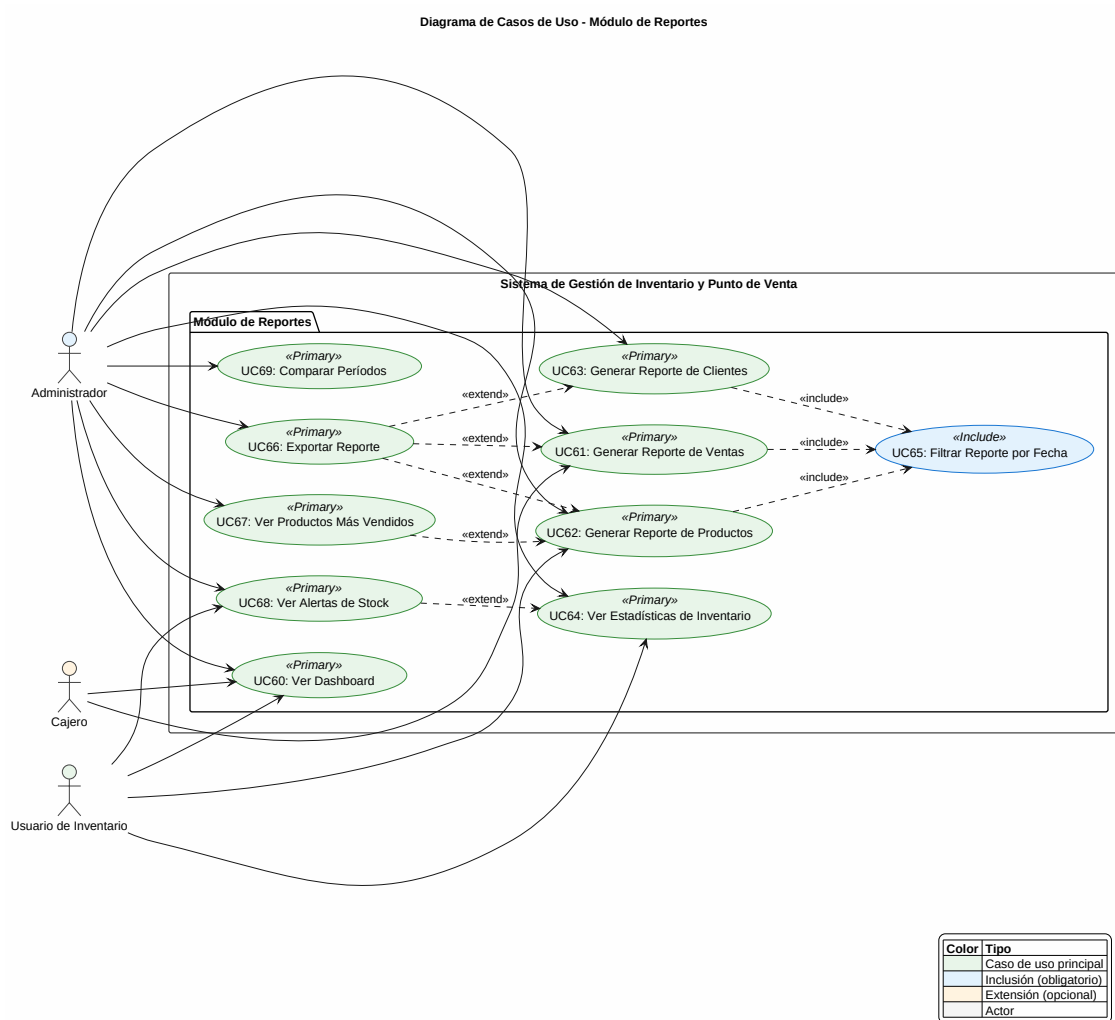


Fig. 5 Diagrama de Casos de Uso - Módulo de Reportes

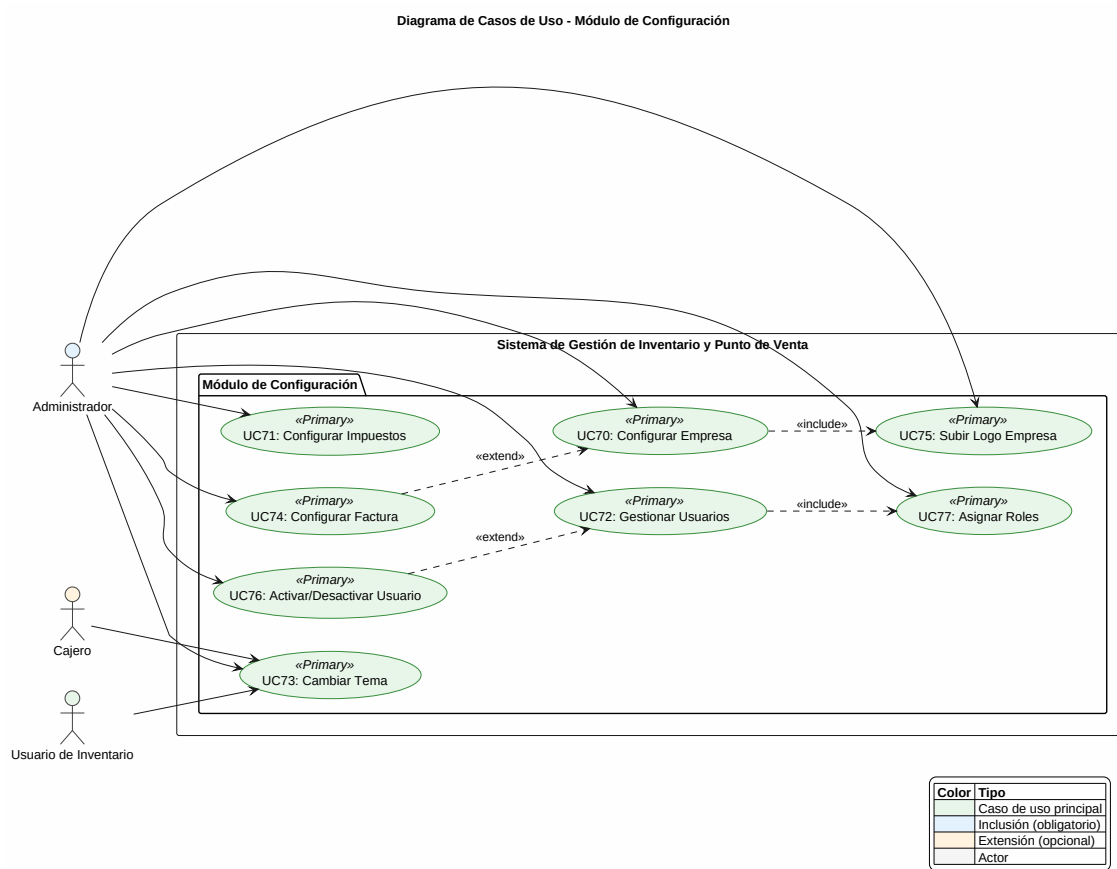


Fig. 6 Diagrama de Casos de Uso - Módulo de Configuración

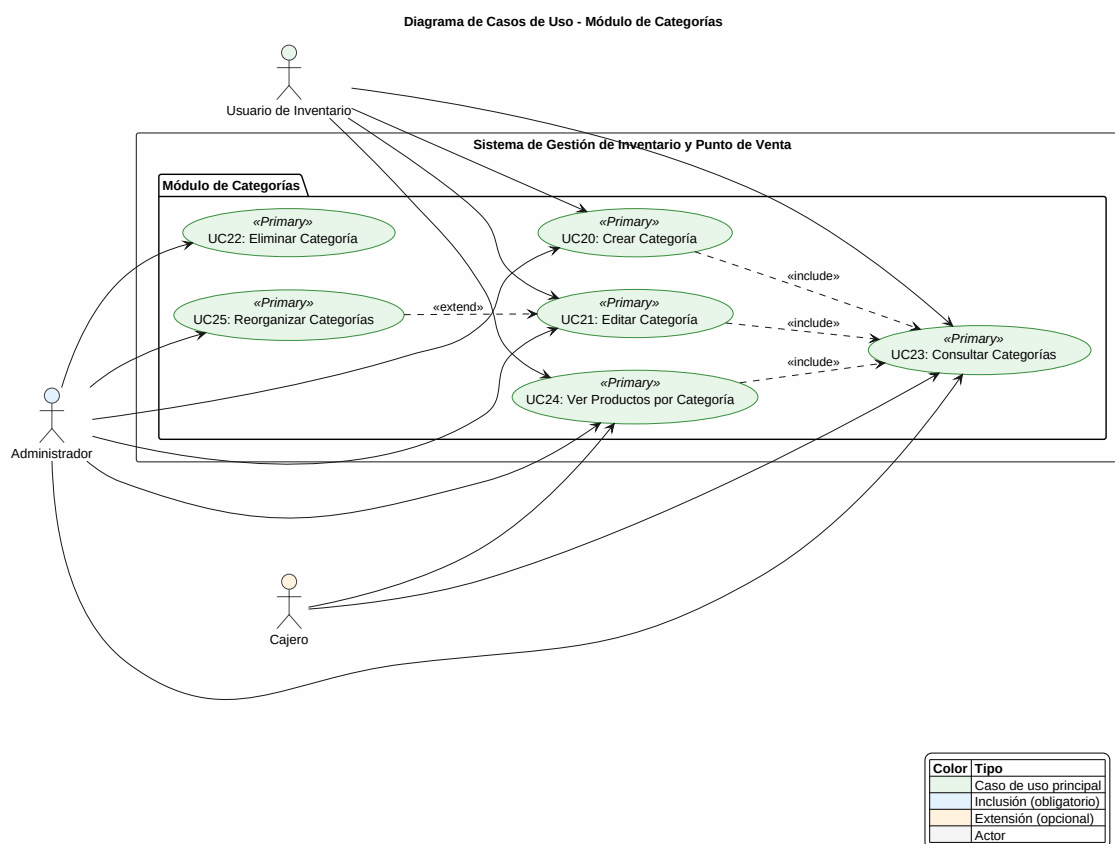


Fig. 7 Diagrama de Casos de Uso - Módulo de Categorías

pueden crear, editar, eliminar y consultar categorías.

B. Diagramas de Componentes

Los diagramas de componentes describen la arquitectura técnica interna del sistema, mostrando cómo se organizan los módulos del backend implementado con NestJS y cómo interactúan entre sí y con las dependencias externas.

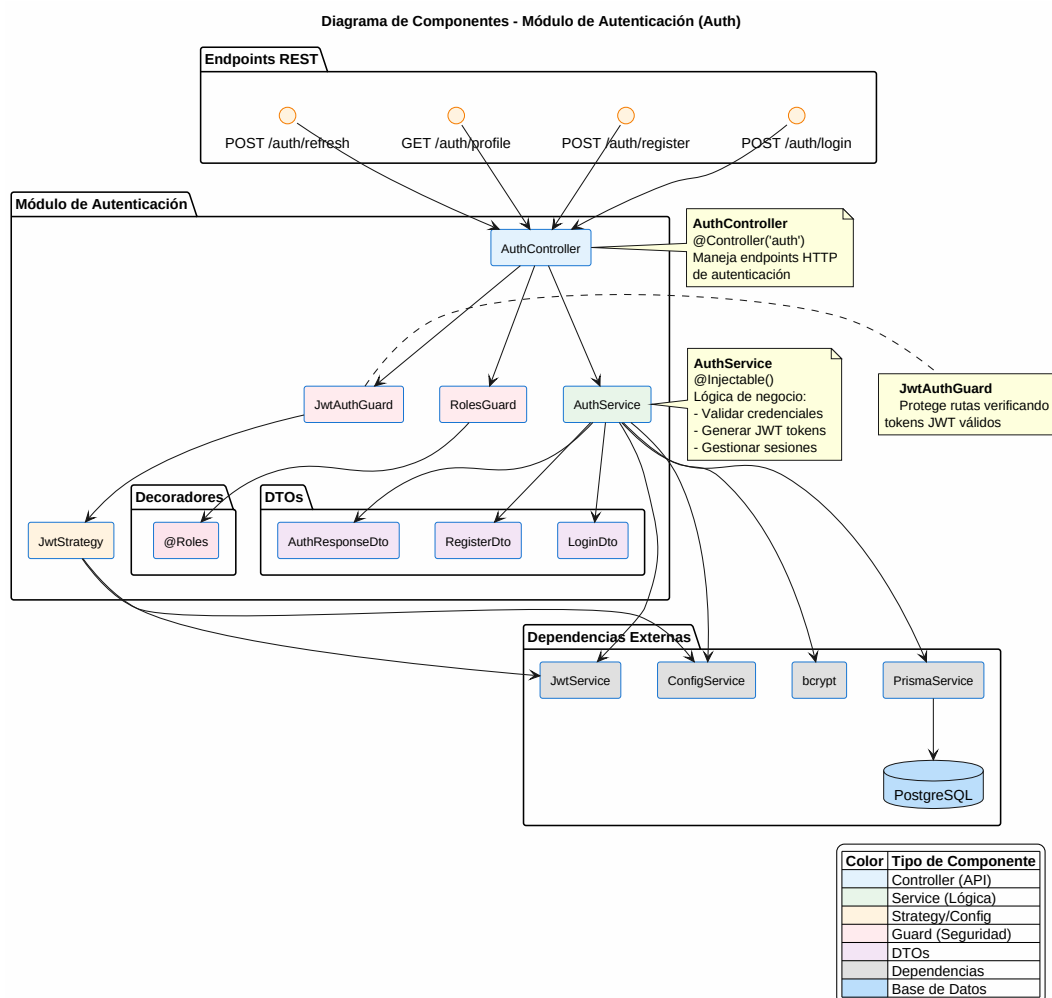


Fig. 8 Diagrama de Componentes - Módulo de Autenticación

El módulo de autenticación (Figura 8) implementa la seguridad del sistema mediante JWT (JSON Web Tokens). El `AuthController` expone los endpoints REST, el `AuthService` contiene la lógica de negocio de validación de credenciales y generación de tokens, los guards (`JwtAuthGuard` y `RolesGuard`) protegen las rutas, y las estrategias (`JwtStrategy`) implementan la validación de tokens. Utiliza `PrismaService` para acceso a datos, `JwtService`

para gestión de tokens, y `bcrypt` para encriptación de contraseñas.

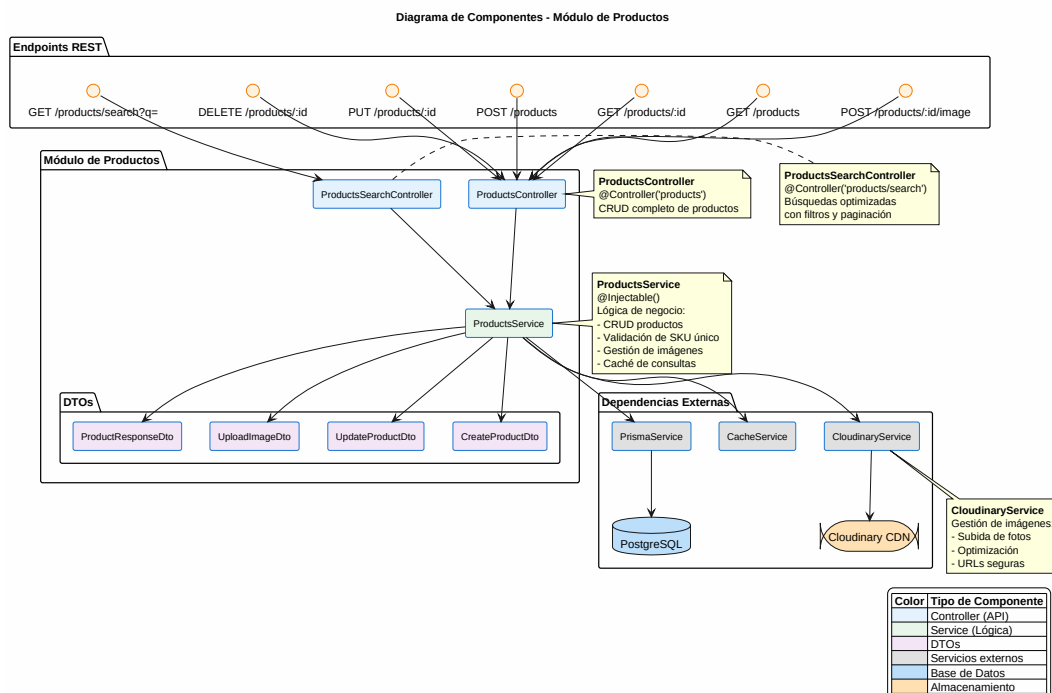


Fig. 9 Diagrama de Componentes - Módulo de Productos

El módulo de productos (Figura 9) gestiona el inventario completo. El `ProductsController` expone endpoints para CRUD de productos, búsqueda y filtrado. El `ProductsService` implementa la lógica de negocio incluyendo control de stock, validaciones, y alertas. Se integra con `Cloudinary` para gestión de imágenes de productos y con `CategoriesService` para organización jerárquica.

El módulo de ventas (Figura 10) implementa el punto de venta con transacciones ACID. El `SalesController` expone endpoints para crear ventas, pausar/reanudar transacciones, y gestionar pagos. El `SalesService` implementa la lógica de negocio con transacciones atómicas que garantizan consistencia de datos. Se integra con `PrismaService` para persistencia y con servicios externos para generación de comprobantes.

El módulo de clientes (Figura 11) permite gestionar la información de clientes. El `CustomersController` expone endpoints para CRUD de clientes, búsqueda y filtrado. El `CustomersService` implementa la lógica de negocio incluyendo historial de compras y soft delete para preservar integridad referencial con ventas históricas.

El módulo de reportes (Figura 12) genera métricas y estadísticas del negocio. El `Report-`

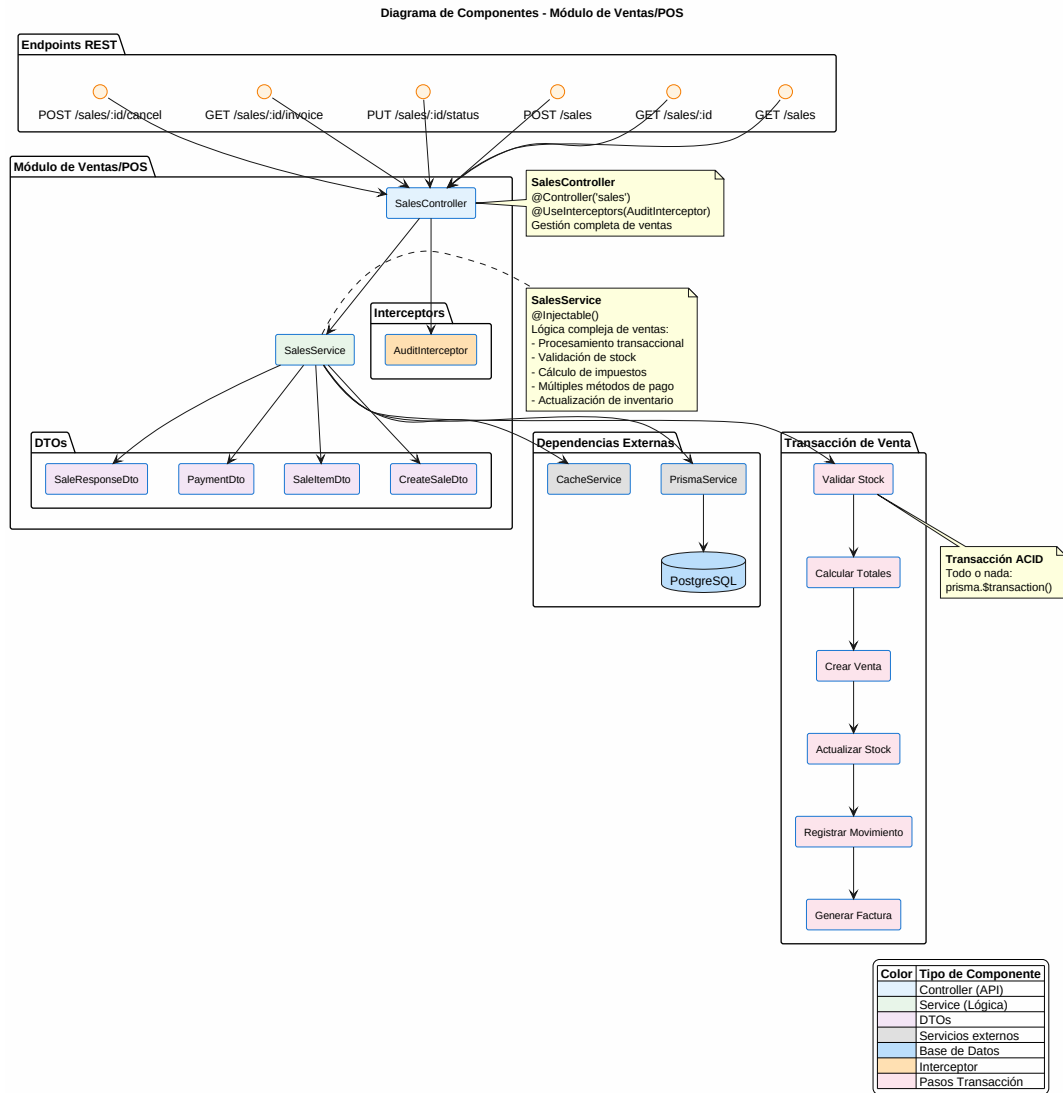


Fig. 10 Diagrama de Componentes - Módulo de Ventas

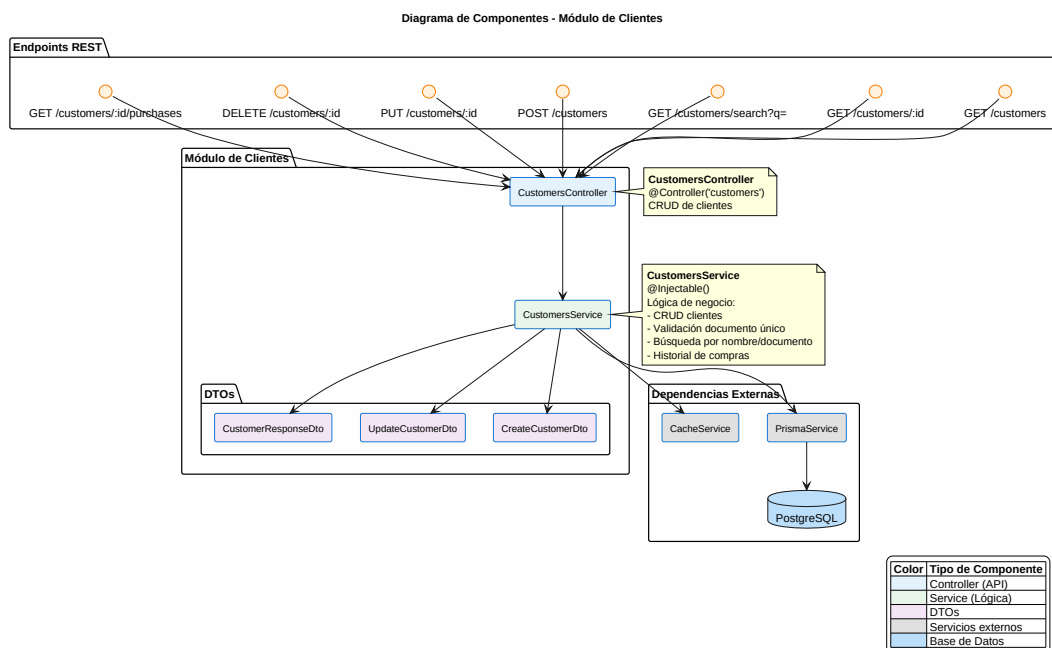


Fig. 11 Diagrama de Componentes - Módulo de Clientes

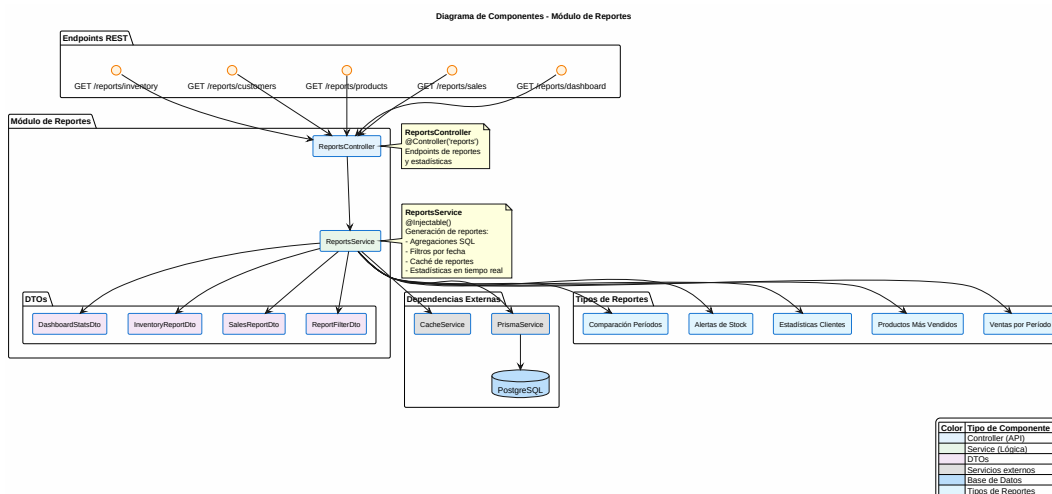


Fig. 12 Diagrama de Componentes - Módulo de Reportes

`tsController` expone endpoints para dashboards, reportes de ventas, productos más vendidos, y tendencias. El `ReportsService` implementa la lógica de negocio con consultas optimizadas y caché para mejorar rendimiento. Se integra con `ExportsService` para generación de reportes en PDF, Excel y CSV.

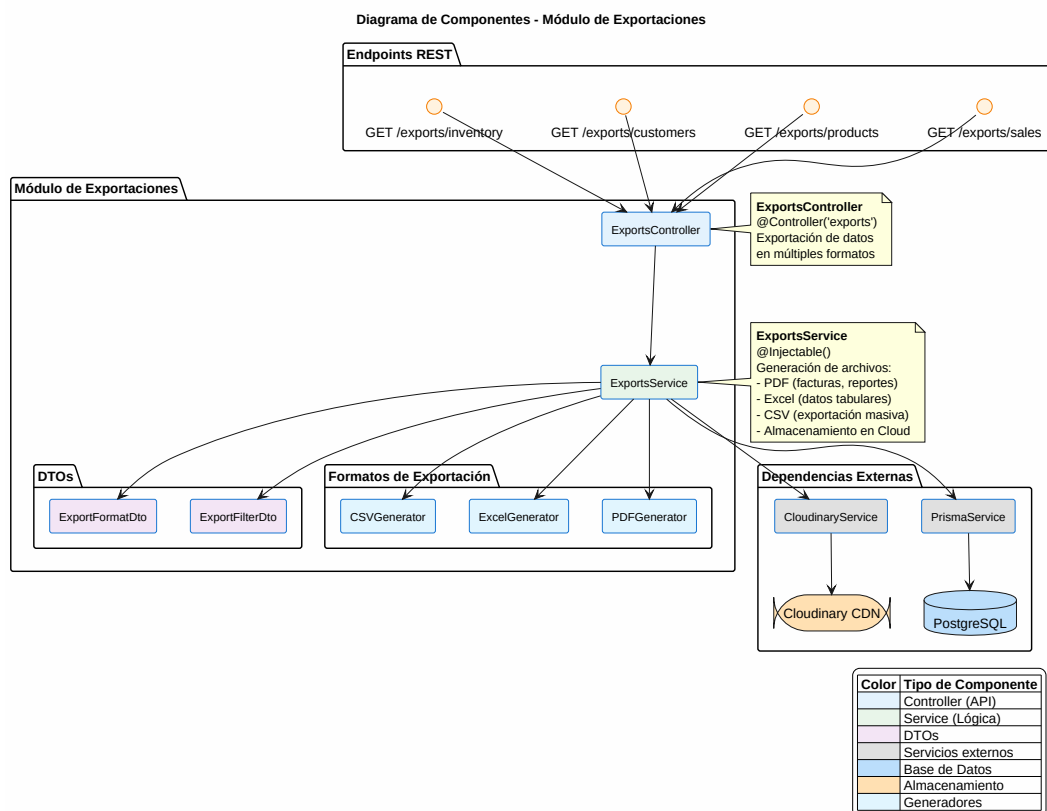


Fig. 13 Diagrama de Componentes - Módulo de Exportaciones

El módulo de exportaciones (Figura 13) permite exportar datos en diferentes formatos. El `ExportsController` expone endpoints para exportar reportes, ventas y productos. El `ExportsService` genera documentos en PDF, Excel y CSV utilizando librerías especializadas.

El módulo de configuración (Figura 14) permite a los administradores gestionar el sistema. El `SettingsController` expone endpoints para gestión de usuarios, roles, y configuraciones del negocio. El `SettingsService` implementa la lógica de negocio incluyendo validaciones de permisos y auditoría.

El módulo de categorías (Figura 15) gestiona la organización jerárquica de productos. El `CategoriesController` expone endpoints para CRUD de categorías y subcategorías. El `Ca-`

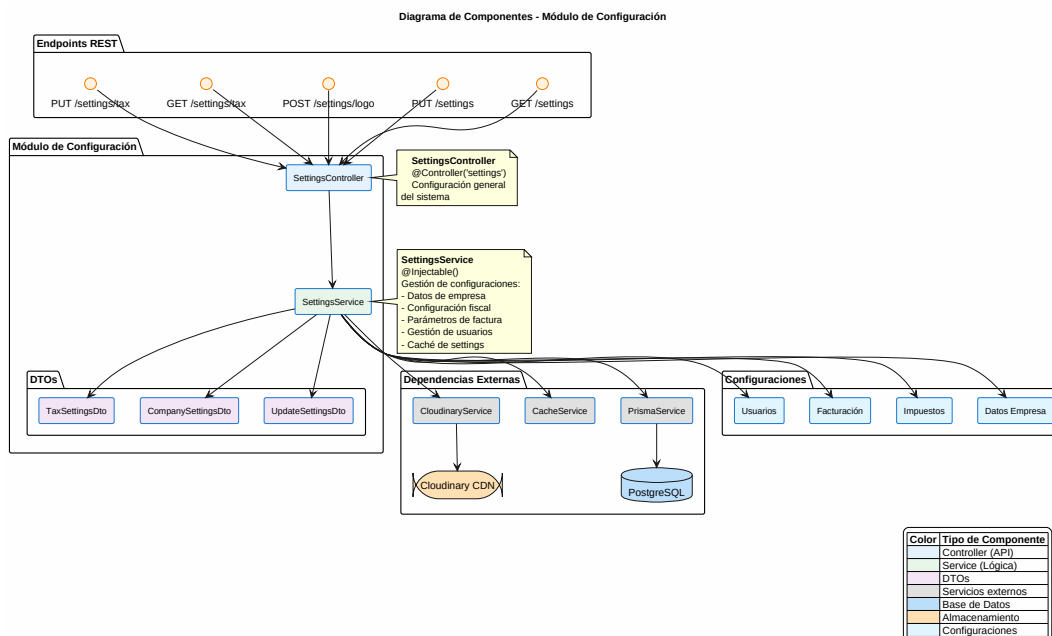


Fig. 14 Diagrama de Componentes - Módulo de Configuración

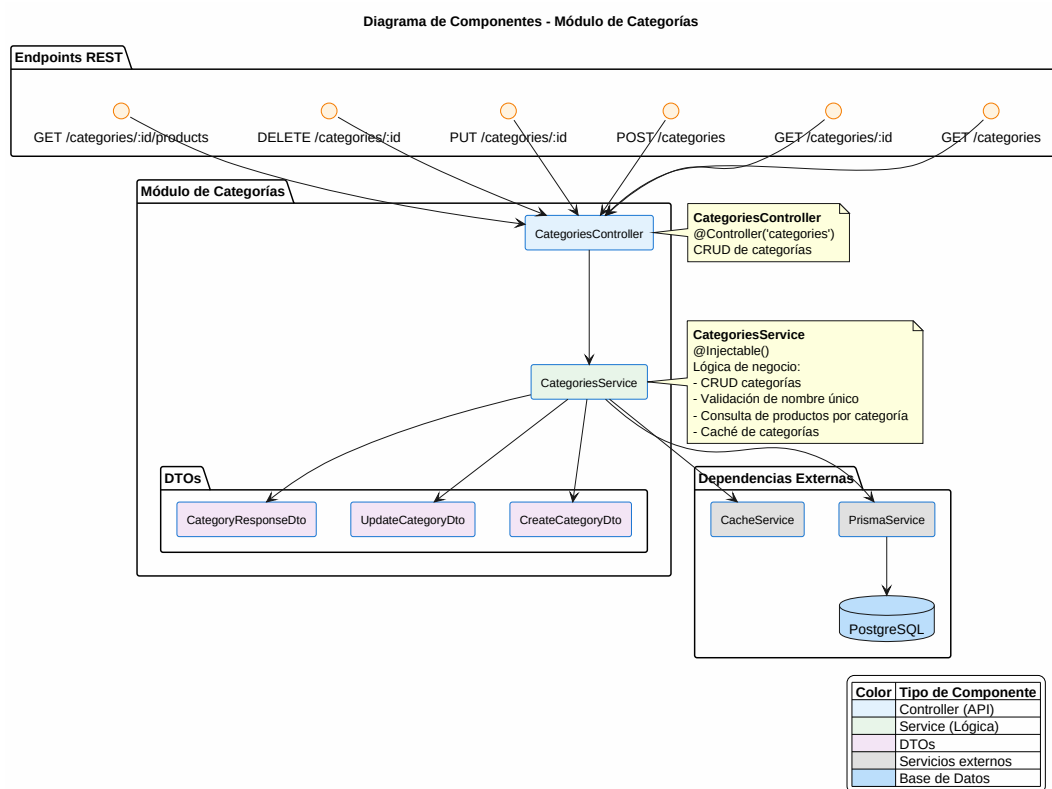


Fig. 15 Diagrama de Componentes - Módulo de Categorías

`categoriesService` implementa la lógica de negocio incluyendo validaciones de integridad referencial.

C. Diagramas de Secuencia

Los diagramas de secuencia muestran el flujo de interacción entre los diferentes componentes del sistema paso a paso, ilustrando cómo se realizan las operaciones críticas del sistema.

El flujo completo de venta (Figura 16) muestra el proceso end-to-end de una transacción de venta, desde que el cajero agrega productos al carrito hasta que se completa la transacción con pago. Este flujo implementa transacciones ACID que garantizan que la venta sea atómica: todas las operaciones se completan exitosamente o ninguna se aplica. Incluye validación de stock, cálculo de totales e impuestos, gestión de múltiples métodos de pago, y generación de comprobante. Si algún paso falla, se ejecuta un rollback automático para mantener la consistencia de datos.

El flujo de autenticación (Figura 17) muestra el proceso de login, generación de tokens JWT, y verificación de permisos. Comienza con el usuario ingresando credenciales en el frontend, que envía una petición POST al backend. El `AuthService` valida las credenciales comparando el password encriptado con `bcrypt`, genera un token JWT con información del usuario y rol, y lo retorna al cliente. Para cada petición posterior, el token se envía en el header `Authorization` y es verificado por el `JwtAuthGuard`. Para operaciones restringidas por rol, el `RolesGuard` verifica si el usuario tiene el permiso necesario antes de permitir el acceso.

El flujo de gestión de productos (Figura 18) muestra cómo se realizan las operaciones CRUD de productos. Incluye validación de datos en dos niveles (frontend y backend), gestión de imágenes mediante `Cloudinary`, actualización de stock en tiempo real, y generación de alertas cuando el nivel de stock cae por debajo del umbral mínimo configurado.

El flujo de generación de reportes (Figura 19) muestra cómo se generan las estadísticas y métricas del negocio. El `ReportsService` ejecuta consultas agregadas optimizadas en la base de datos para calcular KPIs como ventas totales, productos más vendidos, y tendencias temporales. Los resultados se almacenan en caché para mejorar rendimiento en peticiones futuras. Los reportes pueden exportarse en diferentes formatos mediante el `ExportsService`.

El flujo de gestión de usuarios (Figura 20) muestra cómo los administradores pueden registrar nuevos usuarios, asignar roles, y gestionar permisos. Incluye validación de unicidad de email, encriptación de contraseñas con `bcrypt`, y asignación de roles con permisos específicos. Todos los

Diagrama de Secuencia - Flujo de Venta (POS)

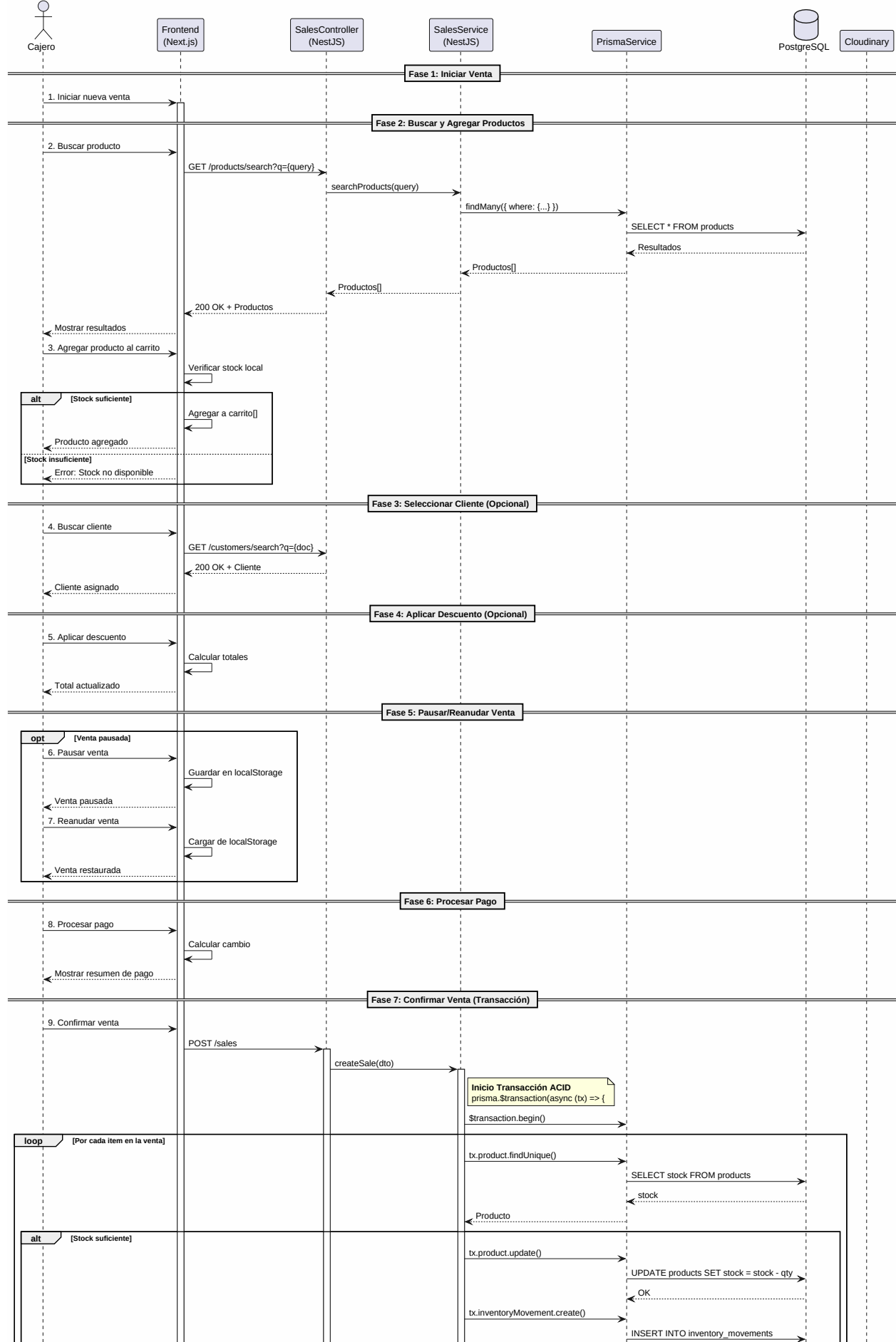


Diagrama de Secuencia - Flujo de Autenticación

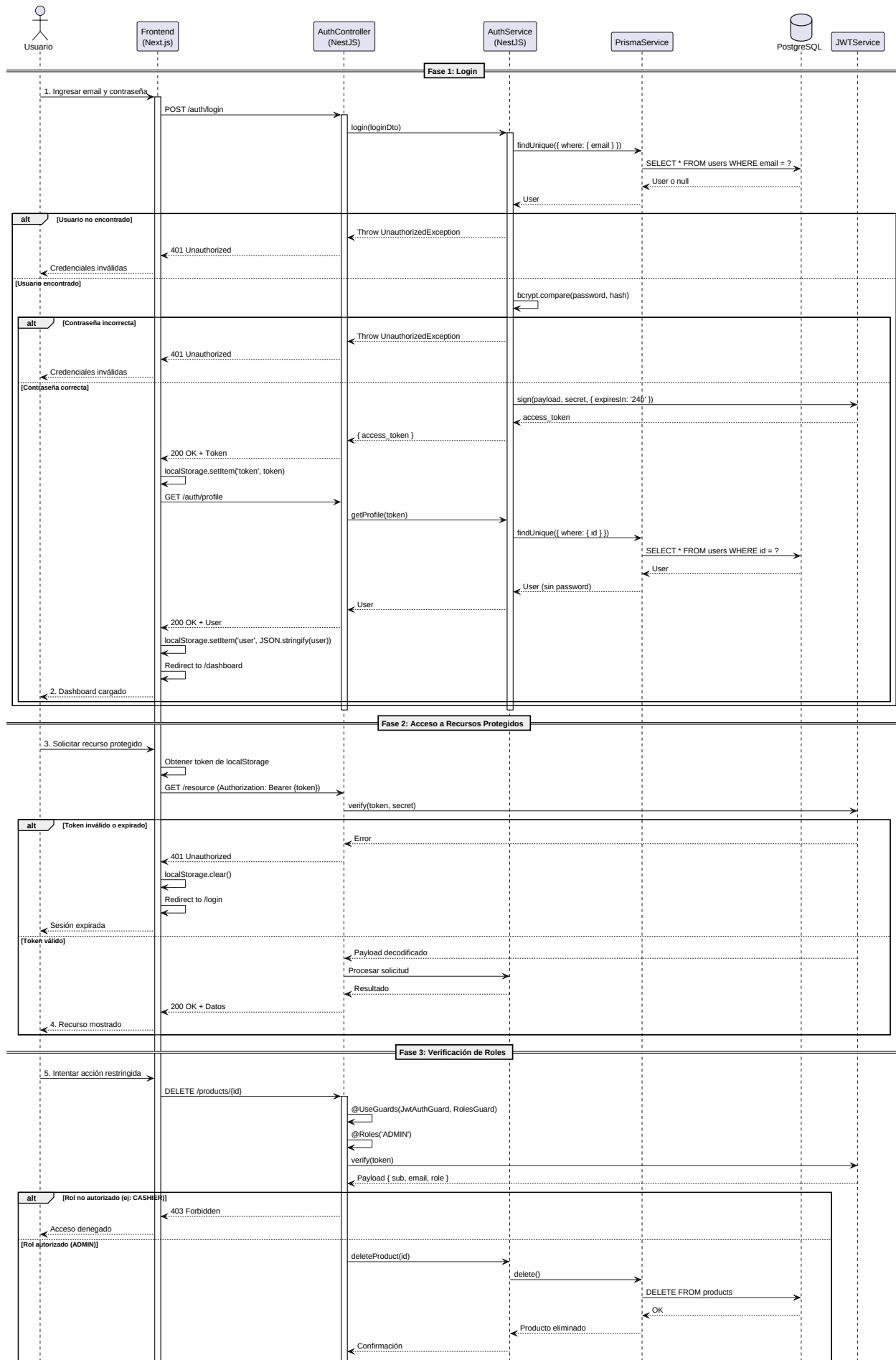
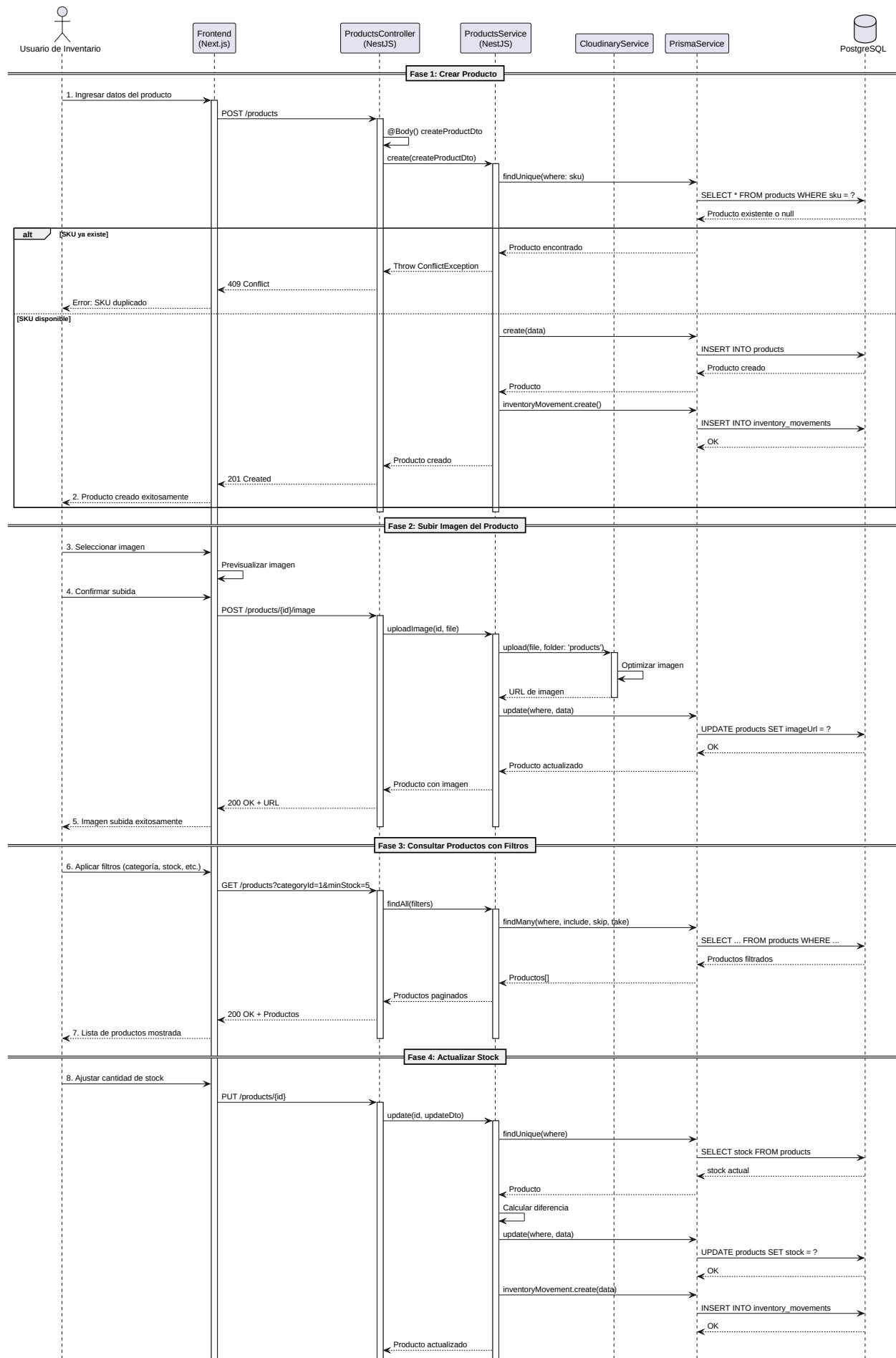


Diagrama de Secuencia - Flujo de Gestión de Productos



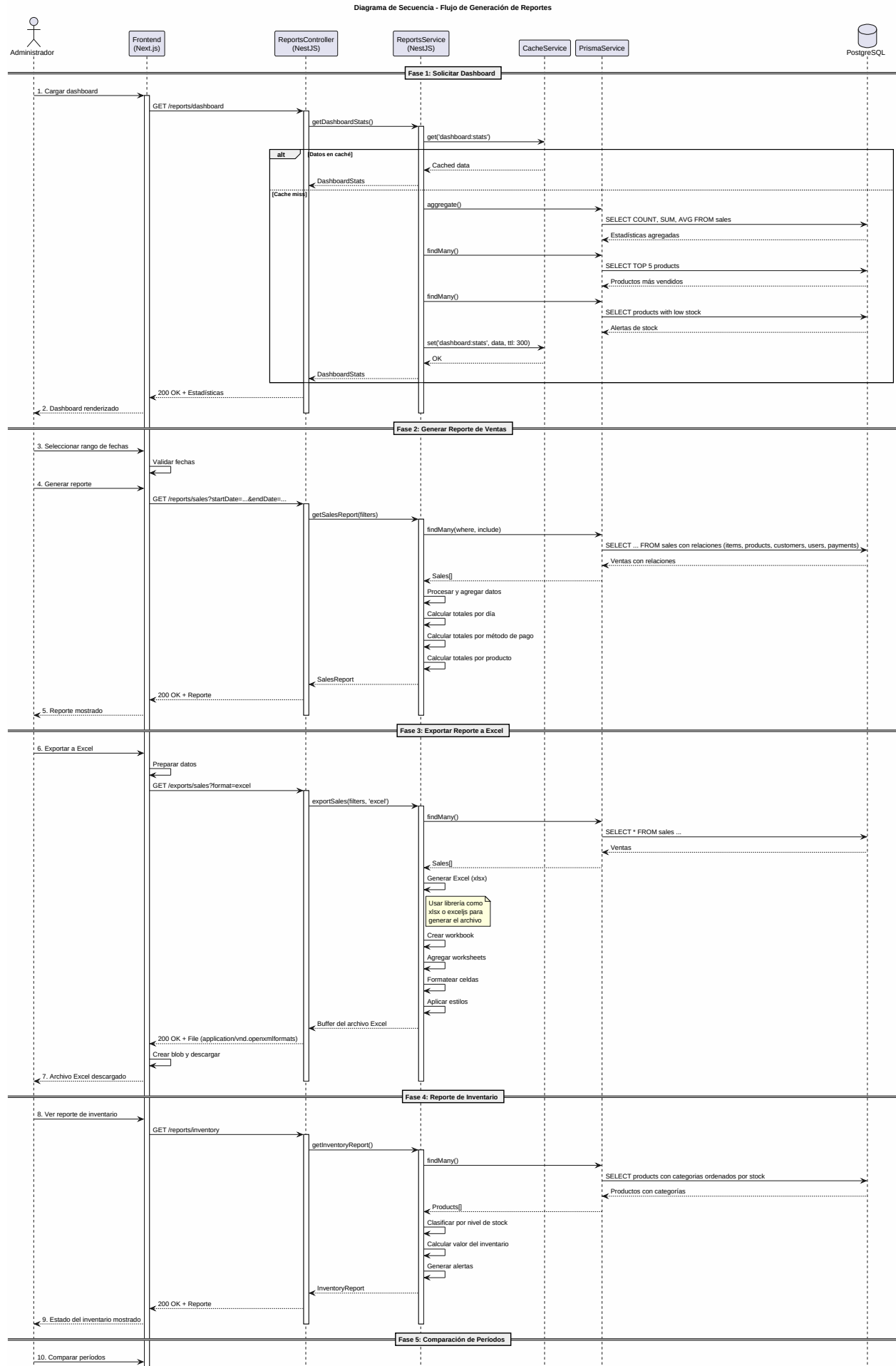
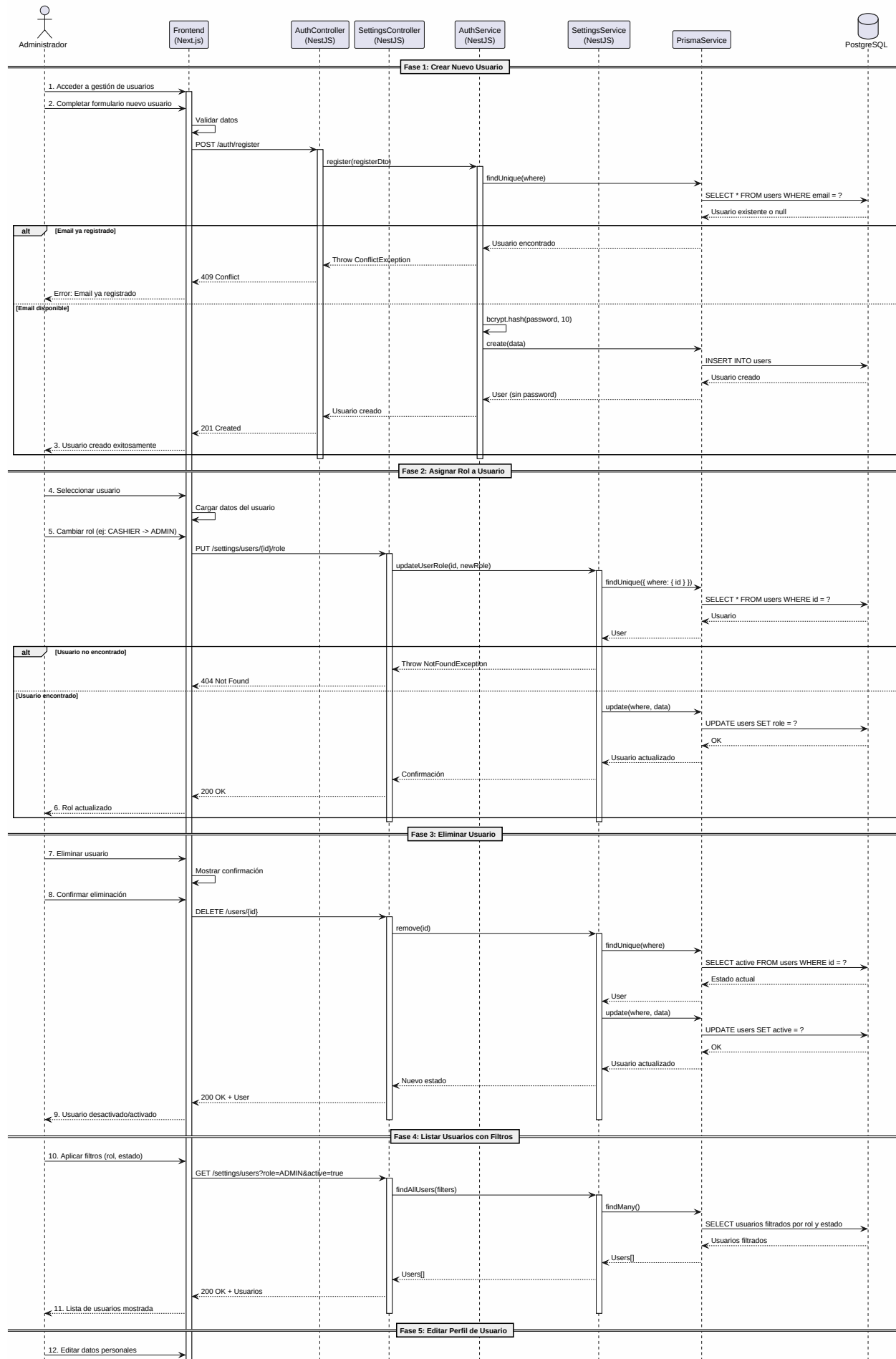


Diagrama de Secuencia - Flujo de Gestión de Usuarios y Permisos



cambios se auditan para mantener trazabilidad completa.

D. Resumen del Diseño

El diseño del sistema se fundamenta en los siguientes principios arquitectónicos:

Arquitectura Modular: El backend está organizado en módulos independientes (auth, products, sales, customers, reports, settings) que pueden desarrollarse, probarse y mantenerse de manera aislada. Cada módulo sigue el patrón Controller-Service-Repository, proporcionando clara separación de responsabilidades.

API REST: La comunicación entre frontend y backend se realiza mediante una API RESTful que utiliza los métodos HTTP estándar (GET, POST, PUT, DELETE) para operaciones sobre recursos identificados con URLs semánticas. La documentación se genera automáticamente mediante Swagger.

Transacciones ACID: Las operaciones críticas como las ventas implementan transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) que garantizan la integridad de datos incluso ante fallos del sistema o errores durante la ejecución.

Seguridad por Roles: El sistema implementa control de acceso basado en roles (RBAC) mediante guards de NestJS. Los roles están claramente definidos (ADMIN, CASHIER, INVENTORY_USER) con permisos específicos para cada módulo.

Optimización de Rendimiento: El módulo de reportes implementa caché de resultados para evitar consultas repetitivas a la base de datos, mejorando significativamente el rendimiento de peticiones frecuentes de métricas y estadísticas.

Auditoría Completa: Todas las operaciones críticas del sistema incluyen logging y auditoría que permiten rastrear quién realizó cada acción, cuándo se ejecutó, y qué cambios se aplicaron, proporcionando trazabilidad completa para control interno y compliance.

Este diseño garantiza un sistema robusto, escalable, mantenible y seguro, apropiado para las necesidades de pequeños negocios que requieren gestión integrada de inventario, ventas, clientes y reportes.

VIII. METODOLOGÍA

El proyecto se desarrollará en 12 semanas con un enfoque iterativo e incremental, adaptado a las condiciones de un trabajo académico individual. La idea es construir el sistema por partes, empezando por lo esencial y agregando funcionalidades en cada iteración, de modo que al final de cada semana se tenga un avance concreto y medible.

A. *Enfoque Metodológico*

Se seguirá un modelo de desarrollo basado en el concepto de Producto Mínimo Viable (MVP). En la práctica, esto significa identificar las funcionalidades que realmente se necesitan para demostrar que el sistema funciona inventario, ventas, clientes y reportes y concentrar todo el esfuerzo en implementarlas bien, sin distraerse con características secundarias que no aportan al objetivo del proyecto.

Para organizar el trabajo se usarán sprints semanales. Cada lunes se definen las tareas de la semana, y cada domingo se revisa qué se logró y qué queda pendiente. El seguimiento de tareas se hará con un tablero Kanban en Trello, donde cada tarea pasa por tres columnas: por hacer, en progreso y terminada. Este esquema es sencillo pero efectivo para un proyecto individual, porque permite detectar rápido si algo se está atrasando.

El desarrollo se guiará por tres principios prácticos. Primero, simplicidad: siempre implementar la versión más directa que cumpla con el requisito, sin agregar complejidad que no se pidió. Segundo, reutilización: aprovechar las librerías y componentes que ya existen en el ecosistema de las tecnologías elegidas (shadcn/ui para interfaz, Prisma para base de datos, class-validator para validación) en lugar de construir todo desde cero. Tercero, integración temprana: conectar el frontend con el backend desde la semana 3, para detectar problemas de comunicación entre capas antes de que sea costoso corregirlos.

La documentación del proyecto de tesis avanzará en paralelo al desarrollo técnico. Cada vez que se complete una fase, se redactará la sección correspondiente del documento. Esto evita la situación común en proyectos de grado de llegar a las últimas semanas con todo el código listo pero sin documento escrito.

B. *Fases del Proyecto*

El proyecto se divide en seis fases. Cada una tiene un objetivo claro, actividades definidas y entregables que permiten medir el avance. Las fases están pensadas para que cada una construya

sobre lo que dejó la anterior.

Fase 1: Análisis, Diseño y Configuración (semanas 1–2, 40 horas). Las primeras dos semanas se dedican a sentar las bases del proyecto antes de escribir código. Esto incluye: revisar documentación de las tecnologías seleccionadas y proyectos similares; definir los requisitos funcionales (qué hace el sistema) y no funcionales (bajo qué condiciones opera); diseñar la arquitectura por capas y el modelo de base de datos con sus entidades, atributos y relaciones; especificar los endpoints de la API REST; crear wireframes en Figma para validar la interfaz antes de implementarla; y configurar los proyectos NestJS y Next.js, Docker para la base de datos local, y el repositorio Git con sus convenciones de commits.

Al final de esta fase se tendrán: documento de requisitos, diagramas de arquitectura y base de datos, especificación de API, wireframes, y un repositorio funcional con ambos proyectos ejecutándose en modo de desarrollo.

Fase 2: Backend Core (semanas 3–5, 64 horas). Aquí se construye toda la lógica del servidor. Se implementan cuatro módulos principales: autenticación (login con JWT, protección de rutas), inventario (CRUD de productos, categorías, movimientos de stock, alertas de inventario bajo), ventas (registro de transacciones, cálculo de subtotales e IVA, métodos de pago) y clientes (registro, búsqueda, historial de compras, eliminación lógica). Se agregan endpoints de reportes que calculan métricas del negocio y generan datos para gráficas. Todo se conecta a PostgreSQL mediante Prisma ORM, se documenta con Swagger y se valida con pruebas unitarias en Jest.

Entregables: API REST funcional, documentación Swagger interactiva, base de datos con migraciones aplicadas y pruebas unitarias automatizadas.

Fase 3: Frontend Core (semanas 6–8, 64 horas). Se construye la interfaz web que usará el propietario del negocio. Incluye: layout general con menú lateral y barra superior; pantalla de login con manejo de sesiones JWT; módulo de inventario con formularios, tablas paginadas, filtros y vista detallada de producto; punto de venta optimizado para registrar ventas rápido (búsqueda de productos, carrito, cálculo automático de totales); módulo de clientes con registro, edición y visualización de historial; dashboard con gráficas (Recharts) y tarjetas de métricas clave; y una biblioteca de componentes reutilizables (botones, inputs, modales, tablas) basados en shadcn/ui.

Entregables: interfaz web responsive integrada con el backend, módulos funcionales operando de punta a punta, y componentes UI reutilizables documentados.

Fase 4: Integración y Pruebas (semanas 9–10, 40 horas). Se verifica que todo funcione junto. Se ejecutan pruebas de integración que recorren flujos completos por ejemplo, realizar una venta desde la búsqueda de producto hasta la generación del comprobante y se prueban casos de uso reales siguiendo guiones paso a paso. Se corrigen los bugs encontrados, priorizando los que bloquean funcionalidad sobre los cosméticos. Se refina la experiencia de usuario (mensajes de error claros, flujos simplificados) y se implementa la generación de comprobantes de venta en formato imprimible. Se optimiza el rendimiento donde sea necesario (índices en la base de datos, paginación, caché).

Entregables: sistema integrado sin errores críticos, reporte de pruebas con resultados, bugs corregidos en Git, y comprobantes de venta funcionales.

Fase 5: Validación y Despliegue (semana 11, 20 horas). Se despliega el frontend en Vercel y el backend en Railway, ambos con planes gratuitos. Se configura la base de datos en producción con las migraciones y datos iniciales de ejemplo. Luego se invita a dos o tres personas representativas del público objetivo (propietarios de pequeños negocios) a usar el sistema en escenarios guiados: registrar productos, hacer ventas, consultar reportes. Se observa cómo interactúan con la herramienta, se les hace un cuestionario breve sobre usabilidad y utilidad, y se corrigen los problemas más urgentes que se identifiquen.

Entregables: sistema desplegado y accesible por URL, resultados de las sesiones de validación documentados, y ajustes implementados según el feedback recibido.

Fase 6: Documentación Final (semana 12, 20 horas). La última semana se dedica a cerrar la documentación. Se escribe el manual de usuario con capturas de pantalla paso a paso, se completa la documentación técnica (decisiones de arquitectura, estructura del código, proceso de despliegue), se finaliza el documento de tesis integrando los resultados de pruebas y validación, y se prepara la presentación para la defensa con diapositivas que resuman el proyecto.

La estimación de 20 horas semanales se sustenta en la disponibilidad real del estudiante considerando la carga académica del semestre en curso. La distribución propuesta asigna aproximadamente 3 horas diarias de lunes a jueves en horario posterior a clases, 2 horas el viernes destinadas a pruebas y documentación, 4 horas el sábado para sesiones de desarrollo intensivo, y 2 horas el domingo para revisión del avance semanal y planeación de la semana siguiente. Esta dedicación equivale a un régimen de medio tiempo (20 horas sobre 40 horas laborales estándar),

compatible con las demás obligaciones académicas del semestre. El total acumulado de 248 horas en 12 semanas arroja un promedio de 20,7 horas por semana, cifra coherente con la distribución diaria propuesta y con el alcance definido para el proyecto.

TABLA I
Cronograma de Ejecución del Proyecto

Fase	Semanas	Fechas	Horas	Actividades Principales
1. Análisis, Diseño y Configuración	1-2	21 feb – 6 mar 2026	40h	- Revisión bibliográfica - Definición de requisitos - Diseño de arquitectura y BD - Wireframes en Figma - Setup de proyectos - Configuración Docker
2. Backend Core	3-5	7 mar – 27 mar 2026	64h	- Módulo de autenticación - Módulo de inventario completo - Módulo de ventas y clientes - Endpoints de reportes - Documentación Swagger - Pruebas unitarias
3. Frontend Core	6-8	28 mar – 17 abr 2026	64h	- Layout y autenticación - Módulo de inventario (UI) - Punto de venta (POS) - Módulo de clientes - Dashboard y reportes - Componentes reutilizables
4. Integración y Pruebas	9-10	18 abr – 1 may 2026	40h	- Pruebas de flujos completos - Pruebas de integración - Corrección de bugs - Refinamiento UX - Generación de comprobantes
5. Validación y Despliegue	11	2 may – 8 may 2026	20h	- Despliegue en producción - Validación con usuarios - Recopilación de feedback - Ajustes críticos
6. Documentación Final	12	9 may – 15 may 2026	20h	- Manual de usuario - Documentación técnica - Documento de tesis completo - Presentación
TOTAL	12 sem.	21 feb – 15 may 2026	248h	Sistema completo + Documento

TABLA II
Distribución Semanal de Actividades

Día	Horas	Actividad Sugerida
Lunes	3h	Desarrollo backend/frontend
Martes	3h	Desarrollo backend/frontend
Miércoles	3h	Desarrollo backend/frontend
Jueves	3h	Desarrollo backend/frontend
Viernes	2h	Pruebas + documentación
Sábado	4h	Desarrollo intensivo
Domingo	2h	Revisión semanal + planeación
Total	20h	Semana completa

IX. PRESUPUESTO

Se ha establecido un presupuesto enfocado en maximizar el valor obtenido de cada inversión, priorizando el uso de herramientas y tecnologías de código abierto que no requieren licencias costosas.

TABLA III
Presupuesto Estimado del Proyecto

INVERSIONES		
Equipo de cómputo	Laptop/PC para desarrollo (existente)	\$0
Software	Herramientas open-source	\$0
Cloudinary	Plan gratuito (25GB)	\$0
Vercel	Despliegue frontend (gratis)	\$0
Railway	Backend + PostgreSQL (gratis)	\$0
Dominio	Opcional (.com.co)	\$50,000
Subtotal Inversiones		\$50,000

GASTOS DE DESARROLLO		
Internet	3 meses x \$60,000	\$180,000
Electricidad	3 meses x \$30,000	\$90,000
Subtotal Gastos		\$270,000

MANO DE OBRA		
Desarrollador	248 horas x \$15,000/hora	\$3,720,000
Director de tesis	24 horas x \$50,000/hora	\$1,200,000
Subtotal Mano de Obra		\$4,920,000

OTROS GASTOS		
Transporte	Reuniones + validación	\$100,000
Papelería	Impresiones documento	\$30,000
Imprevistos	5 % del total	\$267,500
Subtotal Otros		\$397,500

COSTO TOTAL DEL PROYECTO		\$5,637,500
---------------------------------	--	--------------------

A. Notas sobre el Presupuesto

Inversiones (\$50,000):

- Todas las herramientas de desarrollo son gratuitas (código abierto)
- Servicios de despliegue en planes gratuitos suficientes

- Dominio opcional, puede usarse subdominio gratuito

Gastos de Desarrollo (\$270,000):

- Internet necesario para desarrollo y consultas
- Electricidad por uso del equipo de cómputo

Mano de Obra (\$4,920,000):

- Valor estimado del tiempo invertido por el desarrollador
- Honorarios del director de tesis por asesorías

Ahorro respecto a soluciones comerciales:

- Licencias de software (IDE, herramientas): ~\$2,000,000
- Hosting/servidores comerciales: ~\$600,000/año
- **Total ahorrado: ~\$2,600,000**

X. RESULTADOS

Texto de los resultados

A. Requerimientos del Sistema

Los requerimientos del sistema son esenciales en cualquier proyecto. En el caso del presente software, es importante contar con un conjunto claro de especificaciones que guíen todo el proceso de desarrollo.

B. Requerimientos Funcionales

Los requerimientos funcionales describen las funcionalidades específicas que debe proporcionar el sistema.

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE-QUERIMIENTO:	RF01	Actor	Administrador / Usuario
NOMBRE DEL RE-QUERIMIENTO	Autenticación y control de sesión		
Descripción			
El sistema debe gestionar el acceso seguro de los usuarios mediante credenciales y mecanismos de control de sesión basados en tokens.			
Funcionalidad			
El sistema debe validar las credenciales de acceso (correo y contraseña) contra la base de datos antes de permitir el ingreso del usuario. El sistema debe generar un token de sesión (por ejemplo, JWT) para mantener la autenticación del usuario durante su navegación por el sistema. El sistema debe restringir el acceso a las rutas internas únicamente a usuarios que posean un token de sesión válido y no expirado.			
Criterios de aceptación	1. El sistema debe permitir que el usuario ingrese su correo y contraseña en un formulario seguro (canal cifrado). 2. El sistema debe redirigir al dashboard principal cuando las credenciales sean válidas y el token se genere correctamente. 3. El sistema debe bloquear y redirigir a la pantalla de inicio de sesión cuando se intente acceder a recursos internos sin un token válido.		
Restricciones			
El sistema debe almacenar las contraseñas encriptadas en la base de datos utilizando algoritmos de hashing seguros.			

TABLA IV
RF01: Autenticación y control de sesión

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF02	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Gestión del catálogo de productos		
Descripción			
El sistema debe permitir la administración completa del catálogo de productos comercializados, garantizando la unicidad de identificadores y la disponibilidad inmediata de la información.			
Funcionalidad			
El sistema debe permitir crear, leer, actualizar y desactivar productos, capturando como mínimo: nombre, SKU, código de barras, categoría, precio de costo, precio de venta, impuesto aplicable y stock actual.			
El sistema debe validar que el SKU y el código de barras de un producto sean únicos dentro del catálogo.			
El sistema debe aplicar eliminación lógica (borrado lógico) a los productos con ventas históricas, evitando la pérdida de información asociada.			
Criterios de aceptación	1. El sistema debe mostrar un mensaje de error cuando se intente registrar un producto con un SKU o código de barras ya existente.		
	2. El sistema debe mostrar el producto recién creado o actualizado de forma inmediata en el listado general de inventario.		
	3. El sistema debe impedir la eliminación física de productos que tengan ventas asociadas, permitiendo únicamente su desactivación.		
Restricciones			
El sistema debe manejar paginación en el listado de productos cuando el número de registros supere el límite establecido.			

TABLA V
RF02: Gestión del catálogo de productos

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF03	Actor	Administrador / Usuario de inven- tario
NOMBRE DEL RE- QUERIMIENTO	Registro de movimientos de inventario		
Descripción			
El sistema debe mantener la trazabilidad de las variaciones de inventario distintas a las ventas, mediante el registro estructurado de movimientos de entrada y salida.			
Funcionalidad			
El sistema debe registrar entradas y salidas de inventario clasificándolas por tipo (compra, ajuste, daño, devolución u otros definidos).			
El sistema debe actualizar en tiempo cercano al real el stock disponible del producto según la cantidad registrada en cada movimiento.			
El sistema debe almacenar para cada movimiento la fecha, la hora, el tipo de movimiento, el usuario responsable y una justificación.			
Criterios de aceptación	1. El sistema debe impedir registrar movimientos de salida que dejen el stock resultante en un valor negativo. 2. El sistema debe reflejar inmediatamente en la ficha del producto el nuevo stock disponible tras registrar un movimiento. 3. El sistema debe permitir consultar el historial de movimientos por producto y por rango de fechas.		
Restricciones			
El sistema debe tratar cada movimiento como irreversible una vez confirmado; para corregir errores se debe generar un contra-movimiento con signo contrario.			

TABLA VI
RF03: Registro de movimientos de inventario

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE-QUERIMIENTO:	RF04	Actor	Sistema / Administrador / Usuario de inventario
NOMBRE DEL RE-QUERIMIENTO	Alertas de stock bajo y umbral mínimo		
Descripción			
El sistema debe notificar de forma proactiva sobre productos cuyo nivel de existencias se encuentre por debajo del umbral mínimo configurado, facilitando la toma de decisiones de reabastecimiento.			
Funcionalidad			
El sistema debe permitir configurar un umbral mínimo de stock por producto. El sistema debe identificar y marcar como “stock bajo” los productos cuyo stock actual sea inferior o igual al umbral configurado. El sistema debe resaltar visualmente los productos con stock bajo en el listado de inventario y/o en el dashboard principal.			
Criterios de aceptación	1. El sistema debe recalcular las alertas de stock automáticamente cada vez que se registre una venta o un movimiento de inventario. 2. El sistema debe resaltar claramente los productos en condición de stock bajo mediante iconos, colores o etiquetas diferenciadas. 3. El sistema debe permitir que el usuario modifique el umbral mínimo de un producto y que la alerta se actualice en consecuencia.		
Restricciones			
El sistema no debe generar alertas para productos marcados como inactivos.			

TABLA VII
RF04: Alertas de stock bajo y umbral mínimo

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF05	Actor	Cajero / Administrador
NOMBRE DEL RE- QUERIMIENTO	Procesamiento de venta en tiempo real		
Descripción			
El sistema debe proporcionar una interfaz ágil para la ejecución de ventas, permitiendo la selección de productos, el cálculo automático de totales y el registro de la transacción con impacto inmediato en el inventario.			
Funcionalidad			
El sistema debe permitir buscar productos por nombre, SKU o código de barras y agregarlos a un carrito de venta con la cantidad correspondiente. El sistema debe calcular automáticamente el subtotal, el valor del impuesto y el total final de la venta según los datos del catálogo. El sistema debe registrar la venta con un identificador único, fecha, hora, usuario, productos vendidos, cantidades, totales y cliente asociado (cuando aplique).			
Criterios de aceptación	1. El sistema debe impedir confirmar una venta cuando al menos uno de los productos no disponga de stock suficiente. 2. El sistema debe mostrar al usuario un resumen detallado de la venta antes de su confirmación. 3. El sistema debe completar el registro de la venta y dejarla disponible para consulta en el historial de ventas inmediatamente después de su confirmación.		
Restricciones			
La interfaz de venta debe minimizar el número de clics necesarios para completar una transacción típica en el punto de venta.			

TABLA VIII
RF05: Procesamiento de venta en tiempo real

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF06	Actor	Cajero / Administrador
NOMBRE DEL RE- QUERIMIENTO	Gestión de métodos de pago y cálculo de cambio		
Descripción			
El sistema debe manejar diferentes métodos de pago y calcular el cambio a devolver al cliente cuando el pago se realice en efectivo.			
Funcionalidad			
El sistema debe permitir seleccionar el método de pago para cada venta (efectivo, tarjeta, transferencia u otros definidos).			
El sistema debe permitir ingresar el monto entregado por el cliente en caso de pago en efectivo y calcular automáticamente el cambio a devolver.			
El sistema debe registrar el método de pago y los valores asociados en el detalle de la venta para su posterior análisis.			
Criterios de aceptación	1. El sistema debe validar que el monto entregado por el cliente en efectivo sea igual o superior al total de la venta.		
	2. El sistema debe mostrar de forma clara el valor del cambio a devolver antes de finalizar la transacción.		
	3. El sistema debe permitir filtrar ventas por método de pago en los reportes y consultas.		
Restricciones			
Los métodos de pago disponibles deben ser configurables únicamente por usuarios con permisos de administración.			

TABLA IX
RF06: Gestión de métodos de pago y cálculo de cambio

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF07	Actor	Sistema / Cajero / Administrador
NOMBRE DEL RE- QUERIMIENTO	Actualización automática de stock por venta		
Descripción			
El sistema debe actualizar automáticamente las existencias de los productos involucrados en una venta al momento de confirmar la transacción, manteniendo la consistencia con el módulo de inventario.			
Funcionalidad			
El sistema debe descontar del stock de cada producto la cantidad vendida cuando una venta se confirme exitosamente. El sistema debe registrar un movimiento de inventario de tipo “salida por venta” asociado al identificador de la venta. El sistema debe impedir la confirmación de la venta si, por concurrencia u otros factores, el stock disponible es insuficiente en el momento del cierre.			
Criterios de aceptación	1. El sistema debe mostrar el nuevo stock disponible del producto inmediatamente después de confirmar la venta. 2. El sistema debe impedir la confirmación de la venta si no se logra aplicar correctamente el descuento de inventario. 3. El sistema debe permitir consultar, desde la ficha de un producto, los movimientos de salida generados por ventas.		
Restricciones			
La operación de registro de venta y descuento de inventario debe ejecutarse de forma atómica para evitar inconsistencias.			

TABLA X
RF07: Actualización automática de stock por venta

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF08	Actor	Cajero / Administrador
NOMBRE DEL RE- QUERIMIENTO	Emisión de comprobantes de venta		
Descripción			
El sistema debe generar un comprobante documental de cada venta realizada, disponible en formato imprimible o digital, para soporte del cliente y del negocio.			
Funcionalidad			
El sistema debe generar automáticamente un comprobante al finalizar exitosamente una venta, incluyendo: logo del negocio, fecha, número de venta, datos del cliente (si aplica), detalle de ítems, impuestos, total y método de pago. El sistema debe permitir la visualización del comprobante desde el navegador y su impresión directa o descarga en formato digital (por ejemplo, PDF). El sistema debe manejar un consecutivo incremental y único para la numeración de las ventas.			
Criterios de aceptación	1. El sistema debe generar el comprobante inmediatamente después de finalizar la venta sin requerir acciones adicionales del usuario. 2. El sistema debe presentar el comprobante en un formato compatible con impresoras de tirilla y/o tamaño carta, según configuración. 3. El sistema debe permitir recuperar y volver a imprimir o descargar el comprobante desde el historial de ventas.		
Restricciones			
El diseño del comprobante debe ajustarse a los formatos definidos por la configuración del negocio y las normas vigentes aplicables.			

TABLA XI
RF08: Emisión de comprobantes de venta

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF09	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Gestión de anulaciones y devoluciones		
Descripción			
El sistema debe permitir gestionar anulaciones totales y devoluciones parciales de ventas, ajustando en consecuencia el inventario y los registros históricos.			
Funcionalidad			
El sistema debe permitir seleccionar una venta registrada y marcarla como anulada o registrar la devolución de uno o varios productos.			
El sistema debe generar los movimientos de inventario necesarios para revertir total o parcialmente el descuento de stock asociado a la venta original.			
El sistema debe registrar la fecha, el usuario responsable y el motivo de la anulación o devolución.			
Criterios de aceptación	1. El sistema debe reflejar el ajuste del inventario derivado de la anulación o devolución en el stock de los productos afectados.		
	2. El sistema debe mantener visible en el historial la venta original con un estado que indique “anulada” o “con devolución”.		
	3. El sistema debe impedir la anulación de ventas cuando existan restricciones adicionales configuradas (por ejemplo, cierres de caja o cierres contables).		
Restricciones			
Solo usuarios con permisos de administración pueden ejecutar procesos de anulación o devolución.			

TABLA XII
RF09: Gestión de anulaciones y devoluciones

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF10	Actor	Administrador / Cajero
NOMBRE DEL RE- QUERIMIENTO	Gestión del directorio de clientes		
Descripción			
El sistema debe centralizar la información de los clientes para facilitar su uso en las ventas y en los procesos de seguimiento comercial.			
Funcionalidad			
El sistema debe permitir registrar, editar y buscar clientes, almacenando como mínimo: nombre o razón social, documento de identidad, teléfono, correo electrónico y dirección. El sistema debe validar que no se registren clientes duplicados con el mismo número de documento de identidad. El sistema debe permitir asociar un cliente existente a una venta en el módulo POS y aplicar eliminación lógica cuando se desee inactivar un cliente.			
Criterios de aceptación	1. El sistema debe mostrar un mensaje de error cuando se intente registrar un cliente con un número de documento ya existente. 2. El sistema debe permitir filtrar rápidamente clientes por nombre, documento o correo electrónico. 3. El sistema debe conservar el historial de ventas asociadas al cliente aun cuando este se marque como inactivo.		
Restricciones			
El tratamiento de la información de clientes debe cumplir con las políticas de protección de datos personales aplicables al negocio.			

TABLA XIII
RF10: Gestión del directorio de clientes

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF11	Actor	Administrador / Cajero
NOMBRE DEL RE- QUERIMIENTO	Consulta de historial de compras por cliente		
Descripción			
El sistema debe permitir consultar el historial de compras de un cliente específico, con el fin de apoyar el análisis de comportamiento y la fidelización.			
Funcionalidad			
El sistema debe permitir seleccionar un cliente y listar las ventas asociadas, mostrando fecha, número de venta, monto total y método de pago.			
El sistema debe permitir filtrar el historial de compras por rango de fechas.			
El sistema debe calcular el total acumulado de compras del cliente en el período consultado.			
Criterios de aceptación	1. El sistema debe mostrar únicamente las ventas correspondientes al cliente seleccionado y al período filtrado.		
	2. El sistema debe actualizar el historial de compras en tiempo cercano al real cuando se registre una nueva venta del cliente.		
	3. El sistema debe permitir exportar o imprimir el historial de compras si la funcionalidad de exportación está habilitada.		
Restricciones			
El acceso al historial detallado de compras debe estar restringido a usuarios con permisos autorizados.			

TABLA XIV
RF11: Consulta de historial de compras por cliente

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE-QUERIMIENTO:	RF12	Actor	Administrador
NOMBRE DEL RE-QUERIMIENTO	Dashboard de indicadores y reporte histórico filtrable		
Descripción			
El sistema debe ofrecer una vista ejecutiva del estado del negocio mediante indicadores clave y reportes históricos de ventas con filtros avanzados.			
Funcionalidad			
El sistema debe calcular y mostrar en el dashboard métricas clave como ventas del día, ventas del mes, número de transacciones y número de productos con stock bajo. El sistema debe permitir generar reportes históricos de ventas filtrables por rango de fechas, cliente, producto y método de pago, presentando el detalle de cada transacción. El sistema debe recalcular los totales monetarios del reporte según los filtros aplicados y presentar la información en tablas paginadas y/o gráficas.			
Criterios de aceptación	1. El sistema debe actualizar los indicadores del dashboard en tiempo cercano al real o al recargar la página. 2. El sistema debe permitir al usuario seleccionar un rango de fechas y aplicar filtros combinados para la generación del reporte histórico de ventas. 3. El sistema debe garantizar que los totales mostrados en el dashboard y en los reportes coincidan con la sumatoria de las ventas almacenadas en la base de datos bajo los filtros activos.		
Restricciones			
Las consultas del dashboard y de los reportes deben estar optimizadas para no degradar significativamente el tiempo de respuesta del sistema, aun con volúmenes crecientes de datos.			

TABLA XV
RF12: Dashboard de indicadores y reporte histórico filtrable

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE-QUERIMIENTO:	RF13	Actor	Administrador
NOMBRE DEL RE-QUERIMIENTO	Gestión de categorías de productos		
Descripción			
El sistema debe permitir la creación, modificación, consulta y eliminación de categorías para organizar el catálogo de productos, facilitando la navegación y búsqueda por agrupaciones lógicas.			
Funcionalidad			
El sistema debe permitir crear categorías con nombre único y descripción opcional. El sistema debe permitir editar el nombre y descripción de categorías existentes. El sistema debe permitir desactivar categorías que contengan productos asociados mediante eliminación lógica. El sistema debe mostrar la cantidad de productos asociados a cada categoría en el listado de categorías.			
Criterios de aceptación	1. El sistema debe impedir la creación de categorías con nombres duplicados. 2. El sistema debe reasignar automáticamente los productos a una categoría "Sin categoría" cuando se elimine lógicamente su categoría original. 3. El sistema debe permitir filtrar el catálogo de productos por categoría seleccionada.		
Restricciones			
El sistema debe validar que el nombre de la categoría no exceda los 100 caracteres y que no contenga caracteres especiales no permitidos.			

TABLA XVI
RF13: Gestión de categorías de productos

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF14	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Gestión de imágenes de productos		
Descripción			
El sistema debe permitir la carga, visualización y eliminación de imágenes asociadas a los productos del catálogo, mejorando la identificación visual en el punto de venta y en los reportes.			
Funcionalidad			
El sistema debe permitir cargar imágenes de productos en formatos JPG, PNG o WEBP con un tamaño máximo de 5MB.			
El sistema debe optimizar automáticamente las imágenes cargadas para reducir el tiempo de carga sin pérdida significativa de calidad.			
El sistema debe almacenar las imágenes en un servicio de almacenamiento en la nube (Cloudinary) y guardar únicamente la URL en la base de datos.			
El sistema debe permitir reemplazar la imagen de un producto existente.			
Criterios de aceptación	1. El sistema debe validar el formato y tamaño del archivo antes de iniciar la carga.		
	2. El sistema debe mostrar una vista previa de la imagen antes de confirmar su guardado.		
	3. El sistema debe mostrar la imagen del producto en el módulo de punto de venta y en el comprobante de venta.		
Restricciones			
El sistema debe implementar mecanismos de seguridad para prevenir la carga de archivos maliciosos disfrazados como imágenes.			

TABLA XVII
RF14: Gestión de imágenes de productos

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF15	Actor	Administrador / Cajero
NOMBRE DEL RE- QUERIMIENTO	Búsqueda y filtrado avanzado de productos		
Descripción			
El sistema debe proporcionar capacidades de búsqueda y filtrado avanzadas para localizar productos rápidamente en el catálogo mediante múltiples criterios combinables.			
Funcionalidad			
El sistema debe permitir buscar productos por nombre, SKU, código de barras o categoría de forma individual o combinada.			
El sistema debe implementar búsqueda incremental que muestre resultados mientras el usuario escribe.			
El sistema debe permitir filtrar productos por rango de precios, estado (activo/inactivo) y nivel de stock (normal, bajo, agotado).			
El sistema debe ordenar los resultados por nombre, precio, stock o fecha de creación en orden ascendente o descendente.			
Criterios de aceptación	1. El sistema debe mostrar resultados de búsqueda en menos de 2 segundos para catálogos de hasta 10,000 productos.		
	2. El sistema debe resaltar visualmente los términos de búsqueda coincidentes en los resultados mostrados.		
	3. El sistema debe mantener activos los filtros aplicados al navegar entre páginas de resultados.		
Restricciones			
La búsqueda debe ser insensible a mayúsculas, minúsculas y acentos para mejorar la experiencia del usuario.			

TABLA XVIII

RF15: Búsqueda y filtrado avanzado de productos

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RF16	Actor	Cajero / Administrador
NOMBRE DEL RE- QUERIMIENTO	Aplicación de descuentos en ventas		
Descripción			
El sistema debe permitir aplicar descuentos porcentuales o en valor fijo sobre items individuales o sobre el total de la venta, con el objetivo de facilitar promociones y ajustes comerciales.			
Funcionalidad			
El sistema debe permitir aplicar descuentos de tipo porcentual (%) o valor fijo (\$) a items individuales en el carrito de venta. El sistema debe permitir aplicar un descuento general sobre el subtotal de la venta completa. El sistema debe recalcular automáticamente el subtotal, impuestos y total de la venta al aplicar o modificar descuentos. El sistema debe registrar en el detalle de la venta los descuentos aplicados con su tipo, monto y justificación.			
Criterios de aceptación	1. El sistema debe validar que los descuentos porcentuales se encuentren entre 0 % y 100 %. 2. El sistema debe impedir aplicar descuentos que resulten en un total de venta negativo. 3. El sistema debe mostrar claramente en el comprobante de venta los descuentos aplicados y el ahorro obtenido por el cliente.		
Restricciones			
El sistema debe requerir autorización mediante contraseña de administrador para descuentos superiores al 30 % del valor del ítem o de la venta total.			

TABLA XIX
RF16: Aplicación de descuentos en ventas

Requerimientos funcionales			
Prioridad: Alta			
CÓDIGO DEL RE-QUERIMIENTO:	RF17	Actor	Cajero
NOMBRE DEL RE-QUERIMIENTO	Gestión del carrito de compras temporal		
Descripción			
El sistema debe permitir gestionar items en un carrito de compras antes de confirmar la venta, facilitando modificaciones de cantidad, eliminación de productos y cálculos dinámicos.			
Funcionalidad			
El sistema debe permitir agregar productos al carrito especificando la cantidad deseada. El sistema debe permitir modificar la cantidad de un ítem ya agregado al carrito mediante incremento, decremento o ingreso directo del valor. El sistema debe permitir eliminar items individuales del carrito antes de confirmar la venta. El sistema debe permitir vaciar completamente el carrito para iniciar una nueva transacción. El sistema debe mostrar en tiempo real el subtotal por ítem, el subtotal general, impuestos y total de la venta.			
Criterios de aceptación	1. El sistema debe validar disponibilidad de stock al agregar o modificar la cantidad de un producto en el carrito. 2. El sistema debe solicitar confirmación antes de vaciar completamente el carrito. 3. El sistema debe mantener el carrito activo durante la sesión del usuario hasta que se confirme la venta o se cierre sesión.		
Restricciones			
El sistema debe limitar la cantidad máxima por ítem al stock disponible actual del producto.			

TABLA XX
RF17: Gestión del carrito de compras temporal

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF18	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Consulta de ventas por rango de fechas		
Descripción			
El sistema debe permitir consultar el historial de ventas realizadas filtrando por un rango de fechas específico, facilitando análisis de períodos comerciales y auditorías.			
Funcionalidad			
El sistema debe permitir seleccionar una fecha de inicio y una fecha de fin para delimitar el período de consulta.			
El sistema debe mostrar todas las ventas registradas dentro del rango especificado con su número, fecha, cliente, total y estado.			
El sistema debe calcular y mostrar totales acumulados de ventas, cantidad de transacciones y ticket promedio para el período consultado.			
El sistema debe permitir ordenar los resultados por fecha, número de venta o monto total.			
Criterios de aceptación	1. El sistema debe validar que la fecha de inicio no sea posterior a la fecha de fin. 2. El sistema debe permitir seleccionar rangos predefinidos (hoy, últimos 7 días, último mes, mes actual). 3. El sistema debe permitir exportar los resultados de la consulta en formato Excel o PDF.		
Restricciones			
Las consultas de rangos superiores a un año deben implementar paginación para optimizar el rendimiento del sistema.			

TABLA XXI
RF18: Consulta de ventas por rango de fechas

Requerimientos funcionales			
Prioridad: Baja			
CÓDIGO DEL RE- QUERIMIENTO:	RF19	Actor	Administrador / Cajero
NOMBRE DEL RE- QUERIMIENTO	Registro de múltiples contactos por cliente		
Descripción			
El sistema debe permitir registrar múltiples números telefónicos y direcciones de correo electrónico para un mismo cliente, facilitando la comunicación por diferentes canales.			
Funcionalidad			
El sistema debe permitir agregar múltiples teléfonos para un cliente, clasificándolos por tipo (móvil, fijo, trabajo).			
El sistema debe permitir agregar múltiples correos electrónicos para un cliente.			
El sistema debe permitir marcar un teléfono y un correo como contacto principal.			
El sistema debe validar el formato de teléfonos y correos electrónicos antes de guardarlos.			
Criterios de aceptación	1. El sistema debe permitir registrar al menos 3 teléfonos y 3 correos por cliente.		
	2. El sistema debe impedir guardar teléfonos o correos duplicados para el mismo cliente.		
	3. El sistema debe mostrar los contactos principales destacados visualmente en la ficha del cliente.		
Restricciones			
El sistema debe requerir al menos un método de contacto (teléfono o correo) para poder registrar un cliente.			

TABLA XXII

RF19: Registro de múltiples contactos por cliente

Requerimientos funcionales			
Prioridad: Baja			
CÓDIGO DEL RE-QUERIMIENTO:	RF20	Actor	Administrador
NOMBRE DEL RE-QUERIMIENTO	Segmentación de clientes por criterios		
Descripción			
El sistema debe permitir clasificar y segmentar clientes según criterios comerciales como frecuencia de compra, monto acumulado o fecha de última compra, facilitando estrategias de fidelización.			
Funcionalidad			
El sistema debe calcular automáticamente métricas de comportamiento de compra para cada cliente (total gastado, número de compras, fecha de última compra, ticket promedio). El sistema debe permitir crear segmentos personalizados de clientes basados en rangos de las métricas calculadas. El sistema debe clasificar automáticamente clientes en categorías predefinidas (VIP, frecuente, ocasional, inactivo) según reglas configurables. El sistema debe permitir generar listados de clientes filtrados por segmento o categoría.			
Criterios de aceptación	1. El sistema debe recalcular las métricas de comportamiento cada vez que se registre una nueva venta asociada al cliente. 2. El sistema debe permitir al administrador configurar los umbrales que definen cada categoría de cliente. 3. El sistema debe mostrar visualmente la categoría del cliente en su ficha y en el punto de venta.		
Restricciones			
Los segmentos y categorías deben recalcularse mediante procesos programados para no afectar el rendimiento del sistema durante horarios de operación.			

TABLA XXIII
RF20: Segmentación de clientes por criterios

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF21	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Reporte de productos más vendidos		
Descripción			
El sistema debe generar un reporte que identifique los productos con mayor volumen de ventas en un período determinado, facilitando decisiones de inventario y estrategias comerciales.			
Funcionalidad			
El sistema debe permitir seleccionar un rango de fechas para generar el reporte de productos más vendidos.			
El sistema debe calcular para cada producto la cantidad total vendida y el valor total generado en el período.			
El sistema debe ordenar los resultados por cantidad vendida o por valor total de forma descendente.			
El sistema debe permitir limitar el reporte a un número específico de productos (top 10, top 20, top 50).			
El sistema debe presentar los resultados en formato de tabla y gráfica de barras.			
Criterios de aceptación	1. El sistema debe excluir del reporte las ventas anuladas o devueltas. 2. El sistema debe permitir filtrar el reporte por categoría de productos. 3. El sistema debe permitir exportar el reporte en formato Excel o PDF.		
Restricciones			
El reporte debe considerar únicamente ventas con estado "completada" para garantizar precisión de la información.			

TABLA XXIV
RF21: Reporte de productos más vendidos

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RF22	Actor	Administrador
NOMBRE DEL RE- QUERIMIENTO	Análisis de rentabilidad por producto		
Descripción			
El sistema debe calcular y presentar métricas de rentabilidad por producto comparando precios de costo y venta, facilitando la identificación de productos con mejores márgenes comerciales.			
Funcionalidad			
El sistema debe calcular el margen bruto unitario por producto (precio de venta - precio de costo).			
El sistema debe calcular el margen bruto porcentual por producto ((precio de venta - precio de costo) / precio de venta * 100).			
El sistema debe calcular la rentabilidad total acumulada por producto considerando las cantidades vendidas en un período.			
El sistema debe permitir ordenar productos por margen porcentual o rentabilidad total.			
El sistema debe identificar visualmente productos con márgenes negativos o muy bajos (alertas).			
Criterios de aceptación	1. El sistema debe actualizar los cálculos de rentabilidad cuando se modifiquen los precios de costo o venta de un producto.		
	2. El sistema debe permitir filtrar el análisis por categoría y rango de fechas de ventas.		
	3. El sistema debe presentar un resumen con el margen promedio general del negocio.		
Restricciones			
Los cálculos de rentabilidad deben considerar el precio de costo y venta vigentes al momento de cada transacción registrada.			

TABLA XXV
RF22: Análisis de rentabilidad por producto

Requerimientos funcionales			
Prioridad: Baja			
CÓDIGO DEL RE-QUERIMIENTO:	RF23	Actor	Administrador
NOMBRE DEL RE-QUERIMIENTO	Reporte de ventas por método de pago		
Descripción			
El sistema debe generar un reporte que agrupe las ventas según el método de pago utilizado, facilitando la conciliación financiera y el análisis de preferencias de pago de los clientes.			
Funcionalidad			
El sistema debe permitir seleccionar un rango de fechas para generar el reporte de ventas por método de pago.			
El sistema debe calcular el total de ventas y cantidad de transacciones para cada método de pago (efectivo, tarjeta, transferencia).			
El sistema debe calcular el porcentaje que representa cada método de pago sobre el total de ventas del período.			
El sistema debe presentar los resultados en formato de tabla y gráfica circular (pie chart).			
Criterios de aceptación	1. El sistema debe excluir del reporte las ventas anuladas.		
	2. El sistema debe mostrar el método de pago más utilizado destacado visualmente.		
	3. El sistema debe permitir exportar el reporte en formato PDF o Excel.		
Restricciones			
El reporte debe considerar únicamente ventas confirmadas y no debe incluir transacciones pendientes o en proceso.			

TABLA XXVI
RF23: Reporte de ventas por método de pago

Requerimientos funcionales			
Prioridad: Media			
CÓDIGO DEL RE-QUERIMIENTO:	RF24	Actor	Administrador
NOMBRE DEL RE-QUERIMIENTO	Reporte de movimientos de inventario		
Descripción			
El sistema debe generar un reporte detallado de todos los movimientos de inventario registrados en un período, clasificados por tipo de movimiento, facilitando la auditoría y trazabilidad del stock.			
Funcionalidad			
El sistema debe permitir seleccionar un rango de fechas para generar el reporte de movimientos de inventario.			
El sistema debe permitir filtrar movimientos por tipo (entrada, salida, ajuste, venta) y por producto específico.			
El sistema debe mostrar para cada movimiento la fecha, tipo, producto, cantidad, stock anterior, stock resultante y usuario responsable.			
El sistema debe calcular totales de entradas y salidas por tipo de movimiento para el período consultado.			
Criterios de aceptación	1. El sistema debe ordenar los movimientos cronológicamente del más reciente al más antiguo por defecto.		
	2. El sistema debe permitir filtrar movimientos por categoría de producto.		
	3. El sistema debe permitir exportar el reporte completo en formato Excel incluyendo todos los detalles.		
Restricciones			
El reporte debe incluir únicamente movimientos confirmados y registrados formalmente en el sistema, excluyendo movimientos en borrador o pendientes de aprobación.			

TABLA XXVII
RF24: Reporte de movimientos de inventario

C. *Requerimientos No Funcionales*

Los requerimientos no funcionales desempeñan un papel fundamental en el presente proyecto, ya que definen las características y atributos clave que no están directamente relacionados con la funcionalidad del software, pero que son esenciales para su éxito.

Requerimientos no funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RNF01	Actor	Sistema / Infraestructura
NOMBRE DEL RE- QUERIMIENTO	Protección de credenciales y datos sensibles		
Descripción			
El sistema debe garantizar la confidencialidad e integridad de las credenciales de acceso y de los datos sensibles manejados durante la autenticación y las operaciones principales del sistema.			
Funcionalidad			
El sistema debe utilizar canales de comunicación cifrados, almacenar las credenciales mediante algoritmos de hashing seguros y aplicar mecanismos de sanitización y validación de entradas para prevenir ataques de inyección y ejecución de código malicioso.			
Criterios de aceptación	1. El sistema debe exponer la aplicación únicamente a través de HTTPS y redirigir automáticamente cualquier petición HTTP a HTTPS. 2. El sistema debe almacenar las contraseñas utilizando algoritmos de hashing con sal (por ejemplo, bcrypt o equivalente), sin guardar nunca el texto plano en la base de datos ni en los logs. 3. El sistema debe validar y sanear todos los parámetros de entrada utilizados en consultas a la base de datos o en generación de contenido dinámico, evitando ataques de inyección SQL y XSS.		
Restricciones			
El sistema no debe registrar en texto plano contraseñas, tokens ni información financiera sensible, y debe aplicar consultas parametrizadas en todas las operaciones de acceso a datos.			

TABLA XXVIII
RNF01: Protección de credenciales y datos sensibles

Requerimientos no funcionales			
Prioridad: Alta			
CÓDIGO DEL RE- QUERIMIENTO:	RNF02	Actor	Sistema / Usuario
NOMBRE DEL RE- QUERIMIENTO	Control de acceso y gestión de sesiones de usuario		
Descripción			
El sistema debe gestionar de forma segura el ciclo de vida de las sesiones de usuario, garantizando que solo usuarios autenticados y autorizados accedan a los recursos internos del sistema.			
Funcionalidad			
El sistema debe utilizar mecanismos de autenticación basados en tokens, tiempos de expiración e inactividad, y aplicar control de acceso basado en roles para definir qué acciones puede realizar cada tipo de usuario.			
Criterios de aceptación	1. El sistema debe invalidar automáticamente la sesión del usuario tras un período configurable de inactividad, requiriendo un nuevo inicio de sesión para continuar. 2. El sistema debe rechazar cualquier petición a recursos protegidos cuando el token de sesión esté ausente, sea inválido o esté expirado. 3. El sistema debe permitir asignar y gestionar roles (por ejemplo, administrador, cajero, auxiliar) y restringir las funcionalidades visibles en la interfaz según el rol del usuario autenticado.		
Restricciones			
Los tokens de sesión deben incluir fecha de expiración y no pueden ser reutilizados tras el cierre de sesión o la revocación manual por un administrador.			

TABLA XXIX

RNF02: Control de acceso y gestión de sesiones de usuario

Requerimientos no funcionales			
Prioridad: Alta			
CÓDIGO DEL RE-QUERIMIENTO:	RNF03	Actor	Sistema
NOMBRE DEL RE-QUERIMIENTO	Rendimiento en operaciones críticas de negocio		
Descripción			
El sistema debe ofrecer tiempos de respuesta adecuados en las operaciones críticas del negocio (autenticación, búsquedas, ventas y reportes) para no afectar la operación diaria del pequeño negocio.			
Funcionalidad			
El sistema debe optimizar consultas, aplicar paginación sobre listados extensos y utilizar caché donde sea pertinente, con el fin de mantener tiempos de respuesta dentro de los umbrales establecidos.			
Criterios de aceptación	1. El sistema debe responder en menos de 2 segundos para al menos el 95 % de las operaciones de inicio de sesión, búsqueda de productos en el POS y registro de una venta bajo una carga de hasta 50 usuarios concurrentes. 2. El sistema debe generar reportes de ventas de hasta un mes de datos o 10.000 transacciones en menos de 5 segundos en el entorno de producción objetivo. 3. El sistema debe mostrar indicadores de carga (barras de progreso, spinners o mensajes de “cargando”) cuando una operación supere los 500 ms, evitando que el usuario perciba la interfaz como congelada.		
Restricciones			
Las pruebas de rendimiento deben realizarse en la infraestructura de despliegue definida para el proyecto y utilizando datos de prueba representativos del volumen esperado del negocio.			

TABLA XXX

RNF03: Rendimiento en operaciones críticas de negocio

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RNF04	Actor	Usuario final
NOMBRE DEL RE- QUERIMIENTO	Interfaz intuitiva y de fácil aprendizaje		
Descripción			
El sistema debe proporcionar una interfaz de usuario intuitiva, coherente y fácil de aprender para usuarios con conocimientos básicos de informática, minimizando la necesidad de capacitación extensa.			
Funcionalidad			
El sistema debe disponer de flujos claros para las tareas frecuentes (registro de productos, ventas, consultas) con un número reducido de pasos, mensajes de error comprensibles y elementos de ayuda contextual.			
Criterios de aceptación	1. El sistema debe permitir que un usuario nuevo, tras una inducción de máximo 30 minutos, pueda realizar de forma autónoma una venta completa y consultar el historial de ventas sin asistencia adicional. 2. El sistema debe permitir completar una venta típica (búsqueda de producto, adición al carrito, selección de método de pago y confirmación) en un máximo de 5 pasos interactivos. 3. El sistema debe mostrar mensajes de validación específicos junto a los campos de formularios con errores, indicando claramente qué información debe corregirse.		
Restricciones			
Los textos de la interfaz y los mensajes deben redactarse en un lenguaje claro y consistente, evitando tecnicismos innecesarios para el usuario final.			

TABLA XXXI
RNF04: Interfaz intuitiva y de fácil aprendizaje

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RNF05	Actor	Usuario final
NOMBRE DEL RE- QUERIMIENTO	Adaptabilidad a diferentes dispositivos y tamaños de pantalla		
Descripción			
El sistema debe adaptarse correctamente a diferentes resoluciones y tamaños de pantalla utilizados comúnmente en el entorno del pequeño negocio, garantizando una experiencia de uso adecuada tanto en equipos de escritorio como en dispositivos portátiles.			
Funcionalidad			
El sistema debe aplicar diseño responsive en todas las vistas principales, reorganizando menús, tablas y formularios según el ancho de pantalla disponible y manteniendo niveles mínimos de contraste para una correcta legibilidad.			
Criterios de aceptación	1. El sistema debe visualizar el dashboard, el módulo de ventas y el módulo de inventario sin barras de desplazamiento horizontales en resoluciones entre 1280×720 y 1920×1080 píxeles. 2. El sistema debe adaptar los menús de navegación a un formato colapsable (por ejemplo, menú tipo hamburguesa) en pantallas de menor ancho. 3. El sistema debe cumplir con niveles mínimos de contraste recomendados (equivalentes a WCAG AA) para texto y elementos interactivos en los temas definidos.		
Restricciones			
Las interfaces deben diseñarse siguiendo el sistema de diseño definido para el proyecto, reutilizando componentes UI coherentes en todos los módulos.			

TABLA XXXII

RNF05: Adaptabilidad a diferentes dispositivos y tamaños de pantalla

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RNF06	Actor	Sistema / Equipo de desarrollo
NOMBRE DEL RE- QUERIMIENTO	Escalabilidad de datos y modularidad arquitectónica		
Descripción			
El sistema debe poder crecer tanto en volumen de información como en funcionalidades, sin requerir cambios estructurales profundos en la arquitectura ni afectar significativamente el rendimiento.			
Funcionalidad			
El sistema debe organizarse en módulos lógicos (inventario, ventas, clientes, reportes, seguridad) y utilizar mecanismos de paginación, índices y consultas optimizadas que faciliten el manejo de catálogos y volúmenes de ventas crecientes.			
Criterios de aceptación	1. El sistema debe soportar un crecimiento del catálogo de productos desde 1.000 hasta al menos 20.000 registros mediante ajustes de configuración (índices y paginación), sin modificar la arquitectura base. 2. El sistema debe permitir añadir nuevos módulos de negocio (por ejemplo, proveedores o compras) reutilizando la API y la arquitectura por capas, sin modificar la lógica interna de los módulos existentes salvo puntos de integración definidos. 3. El sistema debe permitir desplegar el frontend y el backend de forma independiente, manteniendo la comunicación a través de interfaces API documentadas y estables.		
Restricciones			
Todos los módulos nuevos deben respetar la arquitectura modular por capas y los contratos definidos en la API, evitando dependencias directas entre la capa de presentación y la base de datos.			

TABLA XXXIII

RNF06: Escalabilidad de datos y modularidad arquitectónica

Requerimientos no funcionales			
Prioridad: Alta			
CÓDIGO DEL RE-QUERIMIENTO:	RNF07	Actor	Sistema
NOMBRE DEL RE-QUERIMIENTO	Integridad transaccional y consistencia de la información		
Descripción			
El sistema debe garantizar que las operaciones críticas que afectan múltiples entidades (por ejemplo, ventas e inventario) conserven la integridad y consistencia de los datos, incluso ante fallos parciales.			
Funcionalidad			
El sistema debe utilizar mecanismos transaccionales de la base de datos para operaciones complejas, aplicando el modelo ACID y restricciones de integridad referencial para evitar inconsistencias lógicas en el modelo de datos.			
Criterios de aceptación	1. El sistema debe asegurar que, si un proceso de venta falla en cualquier punto de la transacción, ni la venta ni los movimientos de inventario asociados queden registrados parcialmente (la operación debe ser atómica). 2. El sistema debe utilizar claves foráneas y restricciones de integridad referencial que impidan eliminar productos o clientes con registros de ventas asociados. 3. El sistema debe conservar íntegramente las transacciones confirmadas tras un reinicio inesperado del servidor de base de datos, sin evidencia de corrupción en las tablas principales.		
Restricciones			
La configuración de la base de datos debe mantener las propiedades ACID, evitando desactivar mecanismos de journal o logging que comprometan la durabilidad e integridad de los datos.			

TABLA XXXIV

RNF07: Integridad transaccional y consistencia de la información

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE-QUERIMIENTO:	RNF08	Actor	Sistema / Infraestructura
NOMBRE DEL RE-QUERIMIENTO	Disponibilidad del servicio y recuperación ante fallos		
Descripción			
El sistema debe mantener una disponibilidad adecuada durante los horarios de operación definidos y contar con mecanismos de recuperación que reduzcan al mínimo el tiempo de interrupción ante fallos.			
Funcionalidad			
El sistema debe desplegarse en una infraestructura que permita la monitorización básica del servicio, el reinicio automático de los procesos de aplicación y la posibilidad de revertir rápidamente a una versión estable en caso de incidentes graves.			
Criterios de aceptación	1. El sistema debe alcanzar al menos un 99 % de disponibilidad durante las pruebas de operación continua en una ventana de al menos una semana. 2. El sistema debe recuperarse automáticamente y volver a estar disponible en menos de 1 minuto tras un fallo simulado del proceso de backend. 3. El sistema debe permitir realizar un rollback a la última versión estable desplegada en un tiempo máximo de 15 minutos desde la detección de un problema crítico en producción.		
Restricciones			
Los mecanismos de alta disponibilidad y recuperación deben configurarse de acuerdo con las capacidades de la plataforma de despliegue utilizada (por ejemplo, servicios PaaS con gestión de versiones y reinicio automático).			

TABLA XXXV
RNF08: Disponibilidad del servicio y recuperación ante fallos

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RNF09	Actor	Equipo de desarrollo
NOMBRE DEL RE- QUERIMIENTO	Mantenibilidad del código mediante modularidad, pruebas y documentación		
Descripción			
El sistema debe ser fácilmente mantenible y extensible por el equipo de desarrollo, reduciendo el riesgo de introducir errores al modificar o agregar funcionalidades.			
Funcionalidad			
El sistema debe estructurar su código en módulos coherentes por dominio, aplicar buenas prácticas de programación y contar con pruebas automatizadas y documentación actualizada de la API y de los componentes principales.			
Criterios de aceptación	1. El sistema debe contar con pruebas unitarias y/o de integración que cubran al menos el 60 % de las funciones de negocio críticas (registro de ventas, movimientos de inventario, cálculo de totales e impuestos). 2. El sistema debe disponer de documentación actualizada de la API (por ejemplo, mediante Swagger u otra herramienta similar), describiendo endpoints, parámetros, tipos de respuesta y códigos de error. 3. El sistema debe permitir realizar cambios en reglas de negocio de un módulo específico (por ejemplo, umbral de stock bajo o reglas de descuento) modificando únicamente los componentes de ese módulo y sus interfaces públicas, sin necesidad de reescribir módulos no relacionados.		
Restricciones			
El código del proyecto debe versionarse en un repositorio Git, desarrollarse principalmente en TypeScript y seguir un conjunto de convenciones de estilo y revisión de código acordadas (por ejemplo, ESLint y revisiones por pares).			

TABLA XXXVI

RNF09: Mantenibilidad del código mediante modularidad, pruebas y documentación

Requerimientos no funcionales			
Prioridad: Media			
CÓDIGO DEL RE- QUERIMIENTO:	RNF10	Actor	Sistema / Administrador
NOMBRE DEL RE- QUERIMIENTO	Auditoría y registro de eventos críticos		
Descripción			
El sistema debe registrar de forma estructurada los eventos críticos y las acciones sensibles realizadas por los usuarios, con el fin de facilitar auditorías, diagnósticos y la detección de comportamientos anómalos.			
Funcionalidad			
El sistema debe generar registros de auditoría para operaciones como inicios de sesión fallidos, cambios de configuración, movimientos manuales de inventario, modificaciones de precios, anulaciones de ventas y cambios de roles de usuario.			
Criterios de aceptación	1. El sistema debe registrar para cada evento crítico la fecha y hora, el usuario responsable, la operación realizada y el identificador del recurso afectado. 2. El sistema debe permitir consultar los registros de auditoría filtrando por rango de fechas, tipo de evento y usuario, ya sea mediante una interfaz administrativa o consultas sobre la base de datos. 3. El sistema debe conservar los registros de auditoría durante un período mínimo de 90 días naturales, salvo que se definan políticas internas más estrictas.		
Restricciones			
Los registros de auditoría no deben incluir información sensible en texto plano (por ejemplo, contraseñas, números completos de tarjetas u otros datos confidenciales), limitándose a identificadores, metadatos y descripciones generales de la operación.			

TABLA XXXVII
RNF10: Auditoría y registro de eventos críticos

D. Ejemplo de una lista

- Item 1
- Item 2

E. Ejemplo de una figura

Texto que explica la figura



Fig. 21 Logo de la UPB

XI. CONCLUSIONES

A partir del trabajo realizado en esta fase de anteproyecto, se presentan las siguientes conclusiones preliminares:

- La revisión del estado actual de los pequeños negocios colombianos confirma que existe una necesidad real de herramientas de gestión accesibles. Muchos de estos negocios siguen dependiendo de registros manuales o aplicaciones aisladas que no se comunican entre sí, lo que genera pérdida de información y dificulta la toma de decisiones. El sistema propuesto apunta directamente a resolver esta fragmentación mediante la integración de inventario, ventas, clientes y reportes en una sola aplicación web.
- El análisis de las soluciones comerciales existentes (Odoo, Alegra, Siigo) permitió identificar que, si bien ofrecen funcionalidades robustas, presentan barreras de costo, complejidad o falta de adaptación al contexto local que limitan su adopción por parte de microempresas. Esto valida la pertinencia de desarrollar una alternativa de código abierto orientada a este segmento.
- La selección tecnológica realizada Next.js para el frontend, NestJS para el backend, PostgreSQL como base de datos y TypeScript como lenguaje transversal se sustenta en criterios verificables de vigencia, adopción en la industria y compatibilidad entre componentes. Todas las tecnologías elegidas cuentan con soporte activo, documentación amplia y comunidades que facilitan la resolución de problemas durante el desarrollo.
- La arquitectura modular por capas definida para el sistema (presentación, lógica de negocio y acceso a datos) proporciona una estructura clara que separa responsabilidades, facilita el mantenimiento y permite que cada capa evolucione de forma independiente. Los patrones de diseño identificados (Repository, Inyección de Dependencias, DTO, Module) son compatibles con el stack seleccionado y resuelven problemas concretos de la implementación.
- La metodología iterativa e incremental planteada, con sprints semanales y una dedicación estimada de 20 horas por semana durante 12 semanas, resulta viable considerando la carga académica del estudiante y el alcance acotado del proyecto. El cronograma con fechas de-

finidas (21 de febrero al 15 de mayo de 2026) establece hitos claros para el seguimiento del avance.

Estas conclusiones corresponden a la etapa de anteproyecto y se actualizarán conforme avance el desarrollo del sistema, incorporando los resultados de la implementación, las pruebas y la validación con usuarios.

BIBLIOGRAFÍA

- [1] Odoo S.A., “Odoo – Open Source Business Apps,” 2024. [En línea]. Disponible en: <https://www.odoo.com>
- [2] Alegra, “Alegra – Software de facturación y contabilidad,” 2024. [En línea]. Disponible en: <https://alegra.com>
- [3] Siigo S.A.S., “Siigo – Software contable y administrativo,” 2024. [En línea]. Disponible en: <https://siigo.com>
- [4] Vercel, Inc., “Next.js Documentation,” 2024. [En línea]. Disponible en: <https://nextjs.org/docs>
- [5] NestJS Contributors, “NestJS Documentation,” 2024. [En línea]. Disponible en: <https://docs.nestjs.com>
- [6] Prisma Data, Inc., “Prisma Documentation,” 2024. [En línea]. Disponible en: <https://www.prisma.io/docs>
- [7] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [En línea]. Disponible en: <https://www.postgresql.org/docs>
- [8] Meta Platforms, Inc., “React Documentation,” 2024. [En línea]. Disponible en: <https://react.dev>
- [9] Microsoft Corporation, “TypeScript Documentation,” 2024. [En línea]. Disponible en: <https://www.typescriptlang.org/docs>
- [10] Cloudinary Ltd., “Cloudinary Documentation,” 2024. [En línea]. Disponible en: <https://cloudinary.com/documentation>
- [11] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Doctoral dissertation, University of California, Irvine, 2000.
- [12] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2015.